

UNIVERSITI TEKNOLOGI MARA

**SMART SCREEN TIME MANAGEMENT
APPLICATION
WITH NETWORK-LAYER ACCESS CONTROL
AND REMOTE ADMINISTRATIVE
MANAGEMENT**

MUHAMMAD AKMAL BIN MOHD ZAID

**BACHELOR OF COMPUTER SCIENCE (HONS.)
COMPUTER NETWORKS**

MARCH 2025

UNIVERSITI TEKNOLOGI MARA

**SMART SCREEN TIME MANAGEMENT
APPLICATION
WITH NETWORK-LAYER ACCESS CONTROL
AND REMOTE ADMINISTRATIVE
MANAGEMENT**

MUHAMMAD AKMAL BIN MOHD ZAID

**Thesis submitted in fulfilment of the requirements
for Bachelor of Computer Science (Hons.)
Computer Networks**

**College of Computing Informatics and Mathematics
(KPPIM)**

MARCH 2025

SUPERVISOR APPROVAL

SCREEN TIME CONTROL APP WITH NETWORK LAYER ENFORCEMENT AND REMOTE ADMIN ACCESS

By

MUHAMMAD AKMAL BIN MOHD ZAID
2024916867

This proposal was prepared under the supervision of the project supervisor Dr. NoorHayati Binti Mohamed Noor. It was submitted to the College of Computing Informatics and Mathematics (KPPIM) and was accepted in partial fulfilment of the requirements for the degree of Bachelor of Computer Science (Hons.) Multimedia Computing.

Approved by



.....

Dr. NoorHayati Binti Mohamed Noor
Project Supervisor

APRIL 22, 2025

STUDENT DECLARATION

I hereby certify this thesis and the project to which it refers are the result of my own efforts. Any idea or quotation from the work of others, whether published or otherwise, is fully acknowledged in accordance with the standard referring practices of the discipline.



.....

Muhammad Akmal Bin Mohd Zaid

2024916867

APRIL 18, 2025

ACKNOWLEDGMENT

First but not least, I would like to thank my supervisor, Dr. Noorhayati Binti Mohamed Noor, with my whole heart and maximum sincerity because of her support in this research and this invaluable guidance which I received during the course of this research. Her unbending support, sound feedback and unending encouragement would be useful in sailing through in this project. Her professional experience and commitment to academic excellence has not just given the path that this study has taken but has also shown a significant impact on my personal attitude towards research and problem solving. It is due to her patient guidance and constructive criticism that the success of this project would have never been achieved.

My parents also deserve a lot of credit since they have loved me and made sacrifices, and they have always believed in me. They have been the most significant source of inspiration, offering me the emotional and moral support required to continue struggling during the hard periods. They have been the motivating force in my academic life as I aspire to achieve success in whatever I do. Lastly, I also wish to thank all my friends and my colleagues who helped me in this research either through technical assistance, moral support or even just by being around so that I can have someone to share the journey. It took the contributions of many people no matter how minor assisting in making this project a success.

ABSTRACT

In the contemporary electronic era, the problem of over consumption of smartphones has become a topic of serious concern especially among the young and the working population. Current screen time management software solutions and programs do not have the enforcement power and real-time tracking that would make them actually useful. Majority of them are application level softwares and can be bypassed easily and are not very functional. The proposed project seeks to design a Smart Screen Time Control App that will have Network Layer Enforcement and Remote Administrative Access. The network level blocking of the Android operating system with VPN technologies will be implemented in the system, whereas the control and communication between the administrator and user will be implemented using Firebase in the real-time mode.

The app keeps track of how much time one spends watching the screen and blocks out internet access automatically when the time limit set by the user is exceeded using the Android VPN service. In addition to conventional blocking, the system has new powerful safety and communication modules. These are an SOS Panic Button that sends instant location-tagged, alerts to the administrator, a Time Extension Request system that enables users to request more screen time and a Tamper Alert system that provides a signal to the administrator about security breaches. A secure web-based dashboard allows the parent or administrator to track the usage patterns through an average weekly horizontal bar chart, control uninstall passwords, and remotely intervene in real-time.

To be a more useful tool in the monitoring of digital behavior, this product is committed to become a reliable and safe screen time control system. The system, with a smart and enforceable platform, should be an incentive to have healthier screen habits, just because it is easier to scan and feel less reliant on a manual check-in.

TABLE OF CONTENTS

CONTENT	PAGE
STUDENT DECLARATION	ii
ACKNOWLEDGMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES.....	ix
LIST OF TABLES.....	xii
LIST OF ABBREVIATION	xiii
CHAPTER 1	1
1.1 Project Background	1
1.2 Problem Statement.....	2
1.3 Project Objectives.....	3
1.4 Project Scope	3
1.5 Significance & Benefits on The Research Project.....	4
1.6 Conclusion.....	5

CHAPTER 2	6
2.1 Overview	6
2.1.1 History of Screen Time Control and Digital Wellbeing	6
2.2 Application of Screen Time Control Systems	8
2.2.1 General Architecture of Screen Time Control Systems.....	8
2.2.2 Existing Research Addressing the Problem.....	9
2.2.3 Focus of the Research Area	10
2.3 Method / Techniques	11
2.3.1 Tools and Techniques Used in Similar Projects	11
2.3.2 Techniques and Enforcement Methods	12
2.3.3 Parameters of Investigation in Screen Time Control Research	13
2.4 Development Tools	15
2.4.1 Android Studio	15
2.4.2 Firebase	16
2.4.3 Kotlin.....	17
2.4.4 Git and Github	17
2.4.5 Figma.....	17
2.4.6 Postman	18
2.5 Related Work Systems	19
2.5.1 Google Family Link	19
2.5.2 Qustodio	20
2.5.3 Net Nanny.....	24
2.5.4 Apple Screen Time	26

2.5.5	Comparison of Systems.....	28
2.5.6	Literature Review Tree.....	29

CHAPTER 3	30
3.1 Project Framework	30
3.1.1 Planning Phase	33
3.1.2 Design Phase	33
3.1.3 Development Phase.....	34
3.1.4 Testing Phase	34
3.2 Proposed Design.....	36
3.2.1 System Architecture Overview.....	36
3.2.2 Use Case	39
3.2.3 Conceptual Diagram.....	40
3.2.4 Physical Diagram.....	41
3.2.5 Flowchart.....	42
3.3 Proposed Development	44
3.3.1 Software Requirements.....	44
3.3.2 Hardware Requirements	46
3.3.3 Libraries.....	47
3.3.4 Android Mobile Application	48
3.3.5 Testing.....	49

CHAPTER 4	50
4.1 Proposed Development.....	50
4.1.1 Admin View (Web Dashboard)	51
4.1.2 User View (Mobile Application)	52
4.2 Testing Results	55
4.2.1 Functional Testing	55
4.2.2 Security and Tamper Resistance Testing.....	63
4.3 Discussion	64
4.4 Summary	65

CHAPTER 5	66
4.1 Conclusion.....	66
4.2 Limitations.....	67
4.3 Recommendations for Future Work	68
REFERENCES	69
APPENDICES.....	72
APPENDIX A: Gantt Chart	72

LIST OF FIGURES

FIGURE	PAGE
2.1 Google Family Link Homepage	20
2.2 Google Family Link Screen Timer	20
2.3 Google Family Link Time Limits	21
2.4 Qustodio Activity Page	22
2.5 Qustodio Screen Time	22
2.6 Qustodio Website	23
2.7 Qustodio Features	23
2.8 Net Nanny Dashboard	24
2.9 Net Nanny Homepage	25
2.10 Net Nanny Screentime Management	25
2.11 Net Nanny Mobile View	26
2.12 Apple Screentime Dashboard	27
2.13 Apple Screentime Summary	27
2.14 Literature Review Tree	29
3.1 Waterfall Diagram	30
3.2 Use Case Diagram	39
3.3 Conceptual Diagram	40
3.4 Physical Diagram	41
3.5 Flowchart	42
3.6 Gantt Chart	53
4.1 Admin Web Dashboard	51
4.2 Mobile Application View	52
4.3 Admin Locked View	53
4.4 Time Limit Reached View	54
4.5 Prayer Time Lock View	54
4.6 Test Case 1 Result	56
4.7 Test Case 2 Result Admin's View	57
4.8 Test Case 2 Result User's View	57
4.9 Test Case 2 Internet Is Blocked	58
4.10 Test Case 3 Result Admin's View	58
4.11 Test Case 4 Result	59
4.12 Test Case 5 Result Admin's View	59
4.13 Test Case 5 Result User's View	60
4.14 Test Case 6 Result Admin's View	60

4.15 Test Case 6 Result User's View	60
4.16 Test Case 7 Admin Setting Up Password for Uninstalling Purpose	61
4.17 Test Case 7 User Entered Wrong Password	61
4.18 Test Case 7 Admin Alerted on the Admin Dashboard	61
4.19 Test Case 8 User Pressed SOS Emergency Button	62
4.20 Test Case 8 Admin Received the Alert on the Admin Dashboard	62

LIST OF TABLES

TABLE	PAGE
2.1 Comparison of systems	28
3.1 Project Framework	31
3.2 User and Admin Functionalities with Tools Used	38
3.3 Software Requirements	45
3.4 Hardware Requirements	46
4.1 Test Cases	56
5.1 Gantt Chart of CSP600	73
5.2 Gantt Chart of CSP650	73

LIST OF ABBREVIATION

API	Application Programming Interface
IDE	Integrated Development Environment
UI	User Interface
UX	User Experience
VPN	Virtual Private Network
FCM	Firebase Cloud Messaging
SDLC	Software Development Life Cycle
OS	Operating System
GDPR	General Data Protection Regulation

CHAPTER 1

INTRODUCTION

In this tech-savvy world, we are truly dependent on smartphones, it keeps us connected, keeps your mind sharper. But too much screen time has resulted in problems such as diminished productivity, lack of restful sleep and even mental health issues. Lots of people especially students and young people have difficulty regulating their own screen time. And, in the meantime, parents and teachers often don't have the means to see or control how devices are used. Most of the screen-time apps on the market are either too rudimentary easy for a child to get around or they don't offer live, dynamic control. However, this gap will be addressed in this work when we provide a system that works for both the user and the system administrator, having stronger enforcement, better security and granting remote access.

1.1 Project Background

Nowadays, smartphones are an everyday thing, and they are something that most people never leave the house without, especially students and employees. Even if they add many benefits to our lives, they have ignited growing concerns that we are bad at managing our time and that we have become overly dependent on digital media. Most of the screen time apps currently available on the Google Play Store fall short most of them work only at the app level, which means screen time limits can easily be bypassed and not effective in limiting usage. Furthermore, these product tools usually provide little real-time monitoring or remote control for parents or educators who want to supervise children's computer usage (Naslund et al., 2023).

With advancements in mobile technology and cloud-based services, there is a growing opportunity to integrate smarter control features into our daily device usage. By combining network-layer enforcement and remote administrative access, this project aims to enhance user discipline and support healthier digital habits. The proposed Smart Screen Time Control App will offer a more robust and tamper-proof solution.

1.2 Problem Statement

1. No Enforceable Screen Time Limitations.

Many of the apps available nowadays like the Digital Wellbeing on Android are only app-level applications. This simplifies avoidance by users since they do not have direct and complete control over the internet and imposing time restrictions. Unless it is enforced strongly, the users especially the children and teenagers are able to indulge in excessive screen time, which leads to distractions, poor digital habits, and reduced productivity. According to the study by Naslund et al. (2023), this unregulated use is directly related to adverse effects on mental well-being.

2. No Real-Time Monitoring or Remote Admin Access.

The existing solutions lack the provisions of real-time remote monitoring on parents or administrators. This invisibility cannot help to intervene early enough, particularly when it comes to educational or parental supervision. In the absence of a safe remote-control system, teachers and parents cannot respond to excessive use until the damage is inflicted. ScienceDirect (2025) also stressed that distant administrative entry is essential to be proactively involved in the case of screen time conduct, especially when teaching in an online or hybrid setting where face-to-face direction is not a possibility.

3. Lack of Tamper-Proof Security and Control.

Many of the screen time applications can be uninstalled or turned off and this is a real danger in high stakes settings. The absence of strong tampering protection makes screen time applications ineffective and unable to deliver a consistent time restriction. Apthorpe et al. (2020) indicated that ill-conceived applications are often associated with privacy and security consequences because of the convenience that the user can appreciate the option to opt-out or avoid the application. In the absence of effective controls, screen time restrictions become more of a guideline, as opposed to an actual limit.

1.3 Project Objectives

1. To identify and implement appropriate Android and network-layer enforcement technologies for restricting screen time usage.
2. To develop a mobile application with real-time screen time tracking, internet access control, and secure remote admin functionalities using Firebase.
3. To evaluate the effectiveness of the system through testing its ability to accurately enforce screen time limits, prevent tampering, and enable responsive remote access.

1.4 Project Scope

This project is about creating a mobile application to assist parents or managers who want to monitor and control their children's or employees' smartphone use. The app is supposed to track how much time people spend on their devices, then automatically turn off their internet access after using up the time that a parent had decided was OK. Contrary to simple screen time tools, this system will leverage Android's VPN capabilities to limit at the network level, so it is more difficult to circumvent and more effective in general.

Administrators will have a standalone dashboard as well, made with Firebase. Parents will be able to see screen time activity in real time and perform actions like locking the device, extending time limits, removing time limits or changing the settings remotely via the dashboard. To make sure the system cannot easily be bypassed, the app includes security protections that prevent you from uninstalling or disabling it. Data recording will include daily usage patterns, block durations, as well as control commands, to analyze usage effectiveness and overall digital wellbeing.

The targeted devices will be Android smartphones, as the project focuses on mobile behavior. The target users include parents, educators, and digital supervisors who want to regulate the digital habits of children, students, or dependents. For this phase, the project will be tested within a small group of users, such as UiTM students or families, to evaluate its performance and effectiveness. To summarise, the scope for this project is:

1. To develop a smart mobile application that collects screen usage data and enforces usage limits through network-layer blocking once the limit is reached. This is in response to the lack of effective screen time control highlighted by Naslund et al. (2023).
2. To build a secure remote admin dashboard that allows real-time monitoring, controlling of screen time, and managing user restrictions using Firebase, as effective supervision tools are missing in most existing applications (ScienceDirect, 2025).
3. To implement tamper-prevention and feedback mechanisms that alert the admin in case of bypass attempts and ensure accurate recording of screen time patterns to support healthier digital habits.

1.5 Significance & Benefits on The Research Project

The project can bring value because it will assist in promoting healthier screen habits with a stronger and more solid control feature that the current apps fail to do very well. With the help of network-level blocking, the app can use network shutouts to disable the internet after the screen time cap has been met, and it is far harder to bypass. Such policing may as well be helpful to parents and other teachers who would wish to ensure that children or students do not spend too much time on their gadgets.

Other features of the project include administrator live remote access, where parents or bosses can view screen time and impose restrictions or adjustments to the system in real time through a secure online dashboard. The apps that are currently available do not enable you to have remote control of the device to this extent and therefore tend to require physical access to the device and minimal administration control.

Another addition to this project is the more formidable monitoring and enforcement that should be tamper resistant. Users are not allowed to uninstall or switch off the app without the approval of an administrator, which brings an additional element of consistent control. Such security is especially significant with a setting such as a school or a household when the focus is not only to send notifications but to go a step further than only that, but also promote healthy screen time usage.

Moreover, the application keeps track of the screen time usage, sessions which have been blocked and any activity of the administrative. This data will be valuable information to the admins to make better decisions about user habits. It is also a feedback system, which is simple to look in and how you live and work like and make positive changes with the right support piece.

In short, it is a project that tries to remove the restrictions of the existing screen time applications through the stricter enforcement, real time access of an administrator, enhanced protections against tampering and more informative usage statistics. These are what make it an effective tool to bearable digital habits and aiding the mental and behavioral wellbeing of users when the world is becoming more and more screen reliant.

1.6 Conclusion

This project purports to help fix what we consider to be the inherent flaws we find in other screen time apps by creating a system that's secure, difficult to cheat, and operates on real-time enforcement. Featuring live monitoring, remote admin access and robust tamper detection, the app is a more trusted way for parents, educators and guardians to manage screen time and foster healthier digital habits. The solution to keep users, students in particular, focused on today's screen-centric world, allowing blocking access to internet when it is time to turn off screen sharing data for intervention.

CHAPTER 2

LITERATURE REVIEW

This chapter explores the related work and existing technologies that address screen time management, digital wellbeing, MDM (Mobile Device Management), and network level- based enforcement. It also points out where current apps fall short, the requirement of remote admin facilities and the importance of privacy and security in app development. In this proposal, we aim to identify the improvements needed in the system, as based on current state-of-the-art methods, in order to lay groundwork for the development system that we are proposing.

2.1 Overview

2.1.1 History of Screen Time Control and Digital Wellbeing

With the increasing proliferation of mobile phones and internet access, digital wellbeing began to receive more attention. Early on, the most tools could do was block some content and password-protect access. But the emphasis gradually shifted to controlling how people could use their devices, that is, what we could do, not the tools themselves. As smartphones and tablets came on the scene, however, the challenge shifted from simply filtering content to comprehending and steering screen time habits. Early parental control apps like Norton Family functioned on the app level, blocking individual services or using simple rules to tell if a kid had been on something he or she should not have been, but they did not go deep enough into the system to provide a full suite of changes.

As concerns mounted about the mental and physical consequences of spending too much time looking at screens like difficulty focusing, disrupted sleep, a sense of social isolation, there was a demand for tools that could do more than block content.

That led companies like Apple and Google to introduce built-in features, like Screen Time on iOS and Digital Wellbeing on Android. Useful as such tools may be, they are out of users' control, and can be circumvented by people who know their way around tech. But most of them also don't have real-time remote access, and they do not enforce rules at the network level, so parents and administrators wind up with precious little control when they really need it.

Newer solutions that hit the market included Artificial Intelligence (AI) and network security concepts as the market began to mature. These modern tools began blending device activity tracking, machine learning, and network-level controls to offer a more complete and intelligent way to monitor user behavior. However, most existing apps still operate at the application level and lack enforcement mechanisms at the OS or network layer, allowing circumvention via private DNS, VPNs, or app uninstallation. This limitation sets the context for this project, which aims to use a deeper system architecture specifically Android's VPN service and Firebase real-time database—to enforce control and allow remote administration.

Naslund et al. (2023) examined the impact of digital technology overuse on the mental health of university students. The study revealed a strong correlation between excessive screen time and negative emotional outcomes such as anxiety, depression, and decreased academic performance. These findings emphasize the need for digital interventions that can limit screen time and promote healthier digital habits.

Mei et al. (2021) focused on the behavioral side of problematic smartphone use. Their research linked screen addiction with impulsivity and poor self-regulation, particularly among adolescents. This suggests that many users are unable to limit their screen time voluntarily, creating the need for automated, externally controlled enforcement.

Sikora (2023) highlighted that most commercial screen time applications are insufficient in practice. Many of these apps do not offer strong enforcement and can be easily turned off or removed. Since they usually work only at the app level and don't manage network access, users can still browse the internet or use other online services even after their screen time limit is up.

2.2 Application of Screen Time Control Systems

2.2.1 General Architecture of Screen Time Control Systems

Screen time control systems are typically designed to consist of three primary components, which include a monitoring module, an enforcement mechanism, and an admin interface. Each of the parts is a significant factor in ensuring that the app is able to trace the usage of the device and provide limitations when needed. Existing systems, such as UsageStatsManager, are used to track screen usage on Android by recording the duration of app run time and screen on time. It gathers useful information, such as the frequency of unlocking the screen, the duration of apps being open and inactive moments.

The enforcement can be done differently based on the degree of absolute the control needs to be. Some of the apps, however, leverage software applications such as AccessibilityService or Device Policy Manager to block apps or lock the screen, but said applications can be more easily turned on or turned off. To entrench the ban further, certain advanced applications invoke the VpnService interface of the Android to prevent the usage of the internet at the network layer. In this method, online material is completely blocked, independently of what the application you are currently using is, further complicating the task of the end user to surmount the blockage by setting up other browsers or covert applications.

The device of the user is connected with a cloud-based dashboard through the admin interface, which is usually created using Firebase Realtime Database and Firebase Authentication. This will enable the parents, teachers or administrators to track the amount of time going through the screens, set limits to usage or go wild by blocking access or freezing the device whenever such actions occur. Faster syncing with Firebase, where a change or a command is made, it is executed immediately. These three components combined make a good ground in the management of digital habits.

2.2.2 Existing Research Addressing the Problem

A number of works have shown that systems of screen time management are on the rise to need to be better performing. Indicatively, Naslund et al. (2023) discovered that there is a strong correlation between excessive screen time and mental health problems among university students; and using the same incorrectly is only a bigger problem when individuals take an excessively long break between switching to new equipment. In the same manner, Mei et al. (2021) measure that screen addiction is an extension of impulsive behavior, which renders controllability difficult in the users. These findings also emphasize the applicability of extra-system interventions in the enforcement of healthy boundaries.

Such applications as Apple Screen Time or Google Family Link have been criticized as not quite reliable to be used in some cases when the person needs control frequently. Apthorpe et al. (2020) observed that some of the parental control applications, and those that can be utilized on the app level in particular, have privacy and security issues. They discovered that user data in some of these apps is not encrypted and thus their sensitive information is exposed. There are also fewer products designed with such features as tamper detection or the blocking of a network that would have made them more reliable.

As Sikora (2023) emphasized, systems of screen time that are capable of intervening at the level of the network, say, in real time are needed to handle the growing process of digital distraction. When other apps promise to keep parents in full control, even most of them still presuppose that the user will keep the rules or will use simple timers and tools strategies that just often fail to do the job of the parent who is in the trenches with a child who may tantrum or has a screen addiction. The project will attempt to overcome such shortcomings, by introducing a system that is based on strong, non-viable enforcement and access by a remote administrator that cannot be avoided easily.

2.2.3 Focus of the Research Area

Studies on screen time management no longer rely on timers. The current literature discusses complete-function digital wellbeing applications that screen, force, and enable control by parents or the administration. Such systems cannot merely track the usage, they need to actively impose time restrictions, preserve the privacy of the users, and enable the centralized control. There is an increasing agreement in both academic and industry circles that surface level controls are inadequate. The tools should be incorporated at both the network and the OS layer in order to provide non-circumventable enforcement.

According to the recent research including the works of ScienceDirect (2025), the granular control mechanisms beyond per-app blocking are required. It is also about the real-time adaptability tools that are supposed to react to the abrupt behavioral variations like binge usage in exams through the means of dynamism in enforcement. Also, in cases of a user reaching his/her limit, it is promoted to give feedback and deliver educational or motivational messages to the user in order to adjust to the healthy mode of behavior.

Moreover, this project is aimed at providing all these features. It also enables real time distance monitoring, real time response to excessive use, and profound enforcement on the network through Android VPN API. Such a combination makes it different and more aligned with the recent research priorities in the sphere of digital wellbeing and cybersecurity.

The remote administration is a key component of the modern time control systems of screen time, particularly to the parents, teachers, or managers of organizations. Through remote access, administrators will be able to adjust and keep track of the usage of the screen-time as well as establish new limits and lock or unlock the device in real time. Absence of this functionality will make the intervention to be delayed and ineffective.

It was also discovered by ScienceDirect (2025) that this is more advantageous to the users in those systems where there is an active engagement of the parent or teachers in the monitoring of the system via cloud-based dashboards. Firebase, as an example, will allow real-time communication and data update between the user device and the administrator, which will facilitate swift and effective control.

2.3 Method / Techniques

2.3.1 Tools and Techniques Used in Similar Projects

Current tools in screen time enforcement fall into two major categories: application-layer control tools and network-layer enforcement tools. Application-layer tools include Google's Digital Wellbeing and Apple's Screen Time. These rely on timers and app usage stats but are limited by their inability to enforce restrictions on web traffic or unknown apps. They are also easily bypassed by clearing app data, uninstalling the app, or resetting settings.

There are more sophisticated tools at the network level. They rely on Android's VpnService API to configure a local VPN, one that is capable of monitoring and blocking internet traffic directly. This allows the app to block internet connectivity without root access or even a persistent online connection. Apps such as SafeSurf, BlockSite and NetGuard work the same way, but they rarely have features for real-time control from an admin dashboard.

When it comes to remote communication, Firebase remains one of the most popular backend-as-a-service (BaaS) in the scene. It takes advantage of Firebase Authentication for secure logins and its Realtime Database can synchronize user data and commands almost immediately. This can be useful for things like sending usage alerts, or remotely issuing block commands.

It also includes security features like tamper-detection services and boot receivers that are baked into it in order to help identify and stop attempts to uninstall the app or force it to close. These tools are also significantly useful when it comes to long-term protection specially for tech-savvy users.

2.3.2 Techniques and Enforcement Methods

Unlike predictive AI-based systems that rely on machine learning models, event-driven logic is largely applied to the enforcement of screen time. The system monitors for certain triggers, such as total screen time or how long particular apps are used, and acts when a set limit is hit. One of the standard ways to place these limits in effect is by shutting off internet access with a virtual private network. This will ensure users cannot load new content when time expires. For example, if the app sees that a user has reached a two-hour time limit, it sends a signal to the backend that turns on the VPN and blocks all outgoing traffic, except for services like the phone, and allows all traffic when a user turns off Screen Time.

Other, more sophisticated apps will use white list or black list systems to be able to filter access, allowing something as crucial as a phone or messaging to function, and then block everything else. Another emerging method is setting up time-based schedule, to limit internet access during certain periods of the day. But when you pair that with other features such as geofencing, you can establish location-based flexible rules such as only providing internet access when the user is at home.

Some others check for tampering using BroadcastReceiver attached to uninstall actions, or detecting roots commencement. If the gate is in anyway tampered with, the app can send an alert directly to the admin dashboard and in certain instances, even restart itself after the user reboots the device.

Finally, Firebase-triggered actions can enable Real-time responses. When the admin updates time limit or blocks the internet, the command is instantly pushed across and app's background service handles all the commands seamlessly in background and time limits gets enforced immediately. While AI might be used in future systems to modify time limits based on user interaction data or calendar events, today most systems based it on condition-laden logic than predictive models.

2.3.3 Parameters of Investigation in Screen Time Control Research

To develop and evaluate screen time control apps, researchers need to determine the important research parameters to help them produce technically effective and user-friendly ones. Blocking reliability which is how well the system is able to control and prevent excessive screen time under varying conditions is one of the most critical factors. A good system can block access to the internet on the device, be it Wi-Fi or mobile data, and prevent users from hopping to other apps or browsers once their time is up. This settled means that even if there is any network change or if a user tries to bypass the restriction the app core function will not be effected.

The tamper resistance is how good the app is at keeping users from force-stopping or uninstalling the app without permission. It's an area where many existing apps fail, as users can usually turn off accessibility services or uninstall the app completely. We could put additional protections into place, for example by making it difficult to remove Ghibli itself through Android's device admin privileges, or boot time receivers that act on attempts to uninstall or reboot. This is particularly useful if the application runs on a child or a student's device, and they attempt to work around limitations.

The time the system takes to respond to the administrative commands is another important metric. Being that the app uses Firebase for remote control, it is also crucial to monitor the speed at which actions like locking internet connection or increasing the screen time are delivered from the admin dashboard to the user's device. Rapid communication also creates a live enforcement, which is crucial when responding to a situation that requires urgent intervention.

The user experience is another important aspect when examining the real effectiveness of a screen time control system. The application should be Seamless on the Background and it should avoid slugging the device or draining the battery too quickly. People are much more likely to want to try, and then disable, an experience that seems buggy or invasive, they added. This is the reason that things like memory usage, battery life and ease the interface feels to use will be perpetually tested when it comes to user testing.

Finally, already mentioned that screen time controls come with a bit of proprietary protection. The app cannot store or share personal data in order to comply with privacy laws like G.D.P.R. Properties With tools like Firebase Authentication, it can safely and securely store data that only the appropriate users can read. The system is also designed to avoid logging personal content, like the messages a child has sent or received, or providing it access to web browsing history, so that it can help enforce screen time limits without invading user privacy.

Overall, the above parameters form a strong reference to determine the success, reliability, and usability of the screen-time control system developed in this work. Solving for each of these foundational elements will allow us to build a platform that is a secure option, while promoting healthier digital habits.

2.4 Development Tools

To develop a screen time control app that is predictable, safe, and responsive, one needs many software development tools each of which have their own role to play in the creation, deployment, and testing of the software. These tools work together to help the app operate efficiently throughout the large array of Android devices, communicate in real-time with the admin dashboard, and effectively apply restrictions at the network level. Here are some of the tools that you used in the project to make this all happen.

2.4.1 Android Studio

The application will be developed primarily on Android Studio, which is the official integrated development environment (IDE) for Android app development developed by Google. Android Studio is designed for making Android app development designed specifically for Android, and with support for programming in either Java or Kotlin, developers have all the options available at their disposal and along with the support of visual layout editor, performance profiling tools and emulator, one of the main features- Android Studio includes. Android Studio is employed for the design and implementation of the user interface, background services and network monitoring components of this project. It grants access to the required Android APIs such as UsageStatsManager, DevicePolicyManager, VpnService etc., which are required for app usage monitoring, device locking, VPN based enforcement. Using the built-in emulator in Android Studio, we can test our application in real time on virtual devices and make sure that the app runs pretty well on different screen sizes and Android versions.

2.4.2 Firebase

Firebase by Google is selected as the cloud backend service in this work. It is an important component that facilitates the communication of the user's device and the remote administrator in real-time. Firebase Realtime Database is especially helpful as it syncs your data between server and client in real-time. Any time an admin allows screen time or pushes a lock command, that update is pushed to the end user's device without delay. For security purposes, the login is served via Firebase Authentication, securely handling who can and cannot access or manipulate the system. This is crucial especially when working on a project that handles sensitive user controls, where security and preservation of data integrity is a must. Optionally, users can deploy a small web dashboard using Firebase Hosting, which would let users like parents or teachers manage settings and view usage from any device with internet access.

2.4.3 Android VpnService

The Android VpnService API is one of the most important ingredients of this solution. It enables the app to declare a virtual private network interface that will automatically route all of the device's internet traffic through the service. In contrast to classical VPNs, VpnService may be set up to block or filter traffic locally on the device. That means you can block off the internet whenever a screen time limit is hit, even if someone tries to work around the restrictions by using private or alternate browser apps. The privacy trade off of third-party VPN providers is avoided, since the VPN is entirely device-local. Plus, it operates without requiring root access, which is a much safer, more universally accessible approach for everyday users.

2.4.4 Kotlin

This project is written in Kotlin, the officially supported computer language for developing Android apps with Google. Kotlin was the selected language over Java because of its modernity syntax, in-built null safety and more performance in the management of background services. It's easier to maintain too, with a concise, readable codebase is the key for apps with many moving parts, from usage tracking to networking, to remote data syncing. Kotlin's coroutines additionally allow you to handle asynchronous tasks, such as sending and receiving data to/from Firebase, without blocking the main UI.

2.4.5 Git and GitHub

Development of this project takes place on Git and is manually synced with GitHub. Git is a distributed version-control system for tracking changes and rolling back to a previous version when necessary and for easing the work of collaborating developers. GitHub builds on this with a cloud platform for hosting repositories, managing issues and working-on feature requests. Both Git and GitHub help to maintain a clean and organized development workflow, as well as reliable backups and streamlined collaboration with code reviews and future updates.

2.4.6 Figma

Figma is used for user interface design and prototype validation. Being a web-based design tool, Figma lets you design user interfaces that are interactive without a single line of code. In being particularly helpful to early design phases, it allows for a quick sketch and usability test of both the mobile app layout and the admin dashboard, for example. Figma also provides real-time collaboration, which can let stakeholders, superiors, or users-to-be can provide the required feedback before development starts.

2.4.7 Postman

Postman is helpful during development and testing to test REST APIs even if external services added, such as push notifications or email alerts down the project pipeline. It gives developers the capability to mimic running API calls to Firebase or others servers from within the app as well as mimic response cases and scenarios to test and ensure the app runs as expected whenever the user is out of internet reach or under unstable network coverage.

In summary, with the incorporation of such tools as Android Studio, Firebase, VpnService, Kotlin, GitHub, Figma and Postman, the foundation for the development of an optimal easy-to-use, efficient and secure screen time blocking application has been established. Its importance is increasing due to the fact that a good MIS allows the project to meet its objectives, in terms of performance and organization. Combined, they enable a nimble and scalable development process that adheres to contemporary practices in mobile app engineering and digital wellbeing offerings.

2.5 Related Work Systems

Various screen time management solutions have been created and released to the market in the past years, providing differently functional, enforcing and targeting different factors. These features are aimed at letting parents, teachers, and organization leaders to better control device time and foster more mindful digital usage. Most popular apps, meanwhile, have the same general set of features for tracking screen time, setting limits on use and blocking specific content, though they differ widely in terms of how well they work, the quality of their user experience and their technical reliability. This chapter review some of the most popular systems and cover how they operate and how do they compare to the approach adopted in this work.

2.5.1 Google Family Link

One of the best-known apps in the category is Google Family Link, a free app for parents to shake off their kids when it's time to put down the screen, which works on Android and Chromebook devices. It sends real-time activity reports, gives parents the ability to approve or block the downloading of new apps from afar, and allows them to set up daily time limits and bed times. But, while it does include some remote-control features, Family Link still leans heavily toward app-layer controls, which aren't all that difficult for kids to circumvent. Even moderately savvy users can reset the device, create guest accounts or turn off accessibility settings to circumvent restrictions. Unlike this project's system, Family Link does not work at the network layer, and lacks VPN-based enforcement, meaning it doesn't have as much capability to really pull the plug on internet access once time limits roll around.

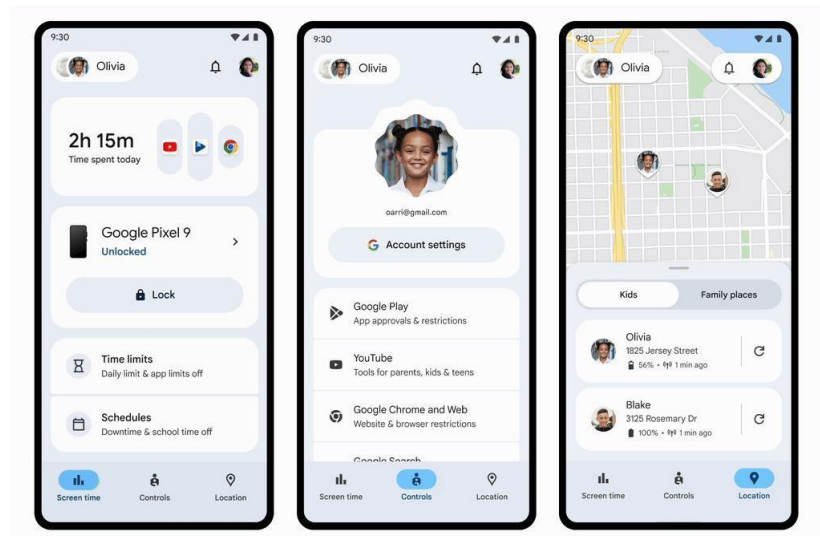


Figure 2.1 Google Family Link Homepage

Source: (*Google*, n.d.)

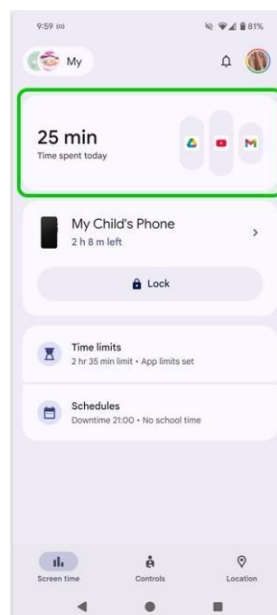


Figure 2.2 Google Family Link Screen Timer

Source: (*Google*, n.d.)

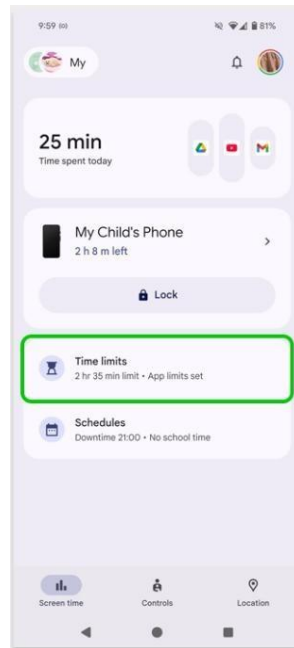


Figure 2.3 Google Family Link Time Limits

Source: (Google, n.d.)

2.5.2 Qustodio

Qustodio is yet another popular parental control option available on various platforms, such as Android, iOS, and Windows. It provides features including app blocking, web filtering, screen time limits and, yes, SMS monitoring. Parents can oversee everything through a dashboard with detailed activity reports, and remote rule settings. Qustodio shines with its detailed reporting, but if you have inquisitive tykes, you might also want a Qustodio Guide and Qustodio control settings that let you configure custom restrictions. But just like Family Link, Qustodio is confined to app- and macOS-level controls and web filtering. It does not use the Android VpnService API to intercept internet traffic in the network stack. In addition, Qustodio could use a decent amount of system resources, and its high level of access may be a potential privacy concern for some users. The demonstrated system, which proposes a light-weight and high-throughput VPN-based blocking system, can overcome these drawbacks, and will not to access the privacy of the user's data.

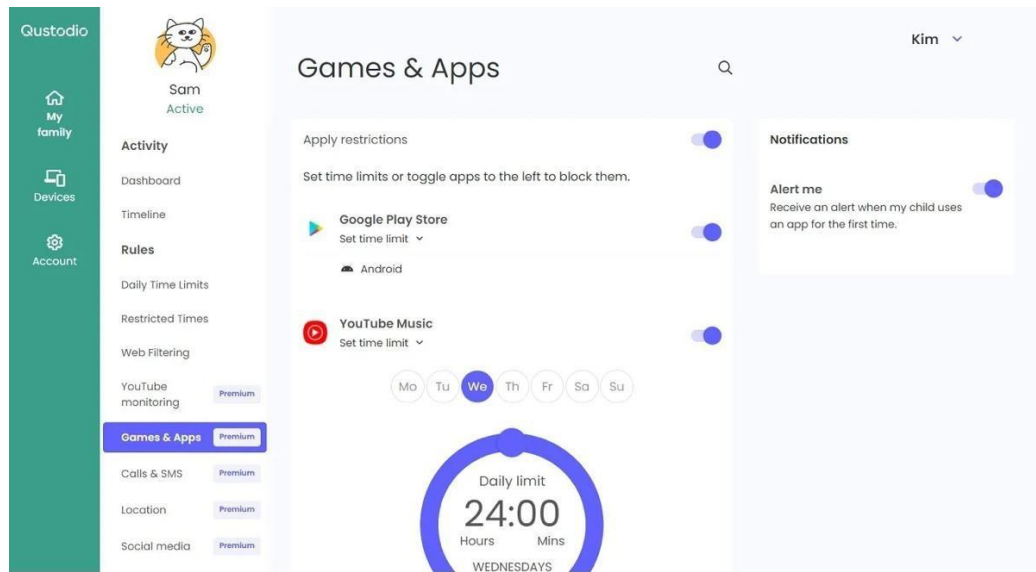


Figure 2.4 Qustodio Activity Page

Source: (Qustodio, n.d.)



Figure 2.5 Qustodio Screen time

Source: (Qustodio, n.d.)

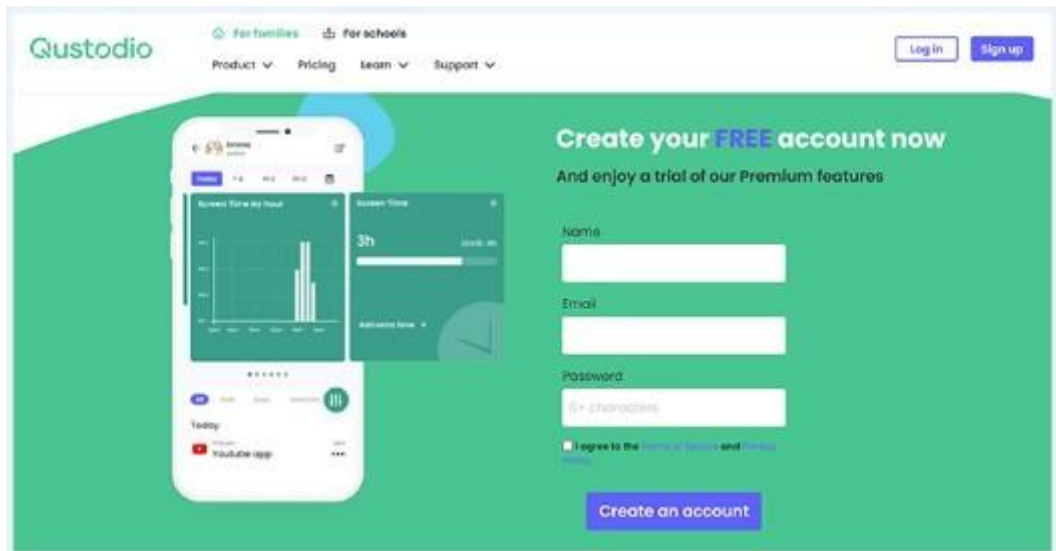


Figure 2.6 Qustodio Website

Source: (Qustodio, n.d.)

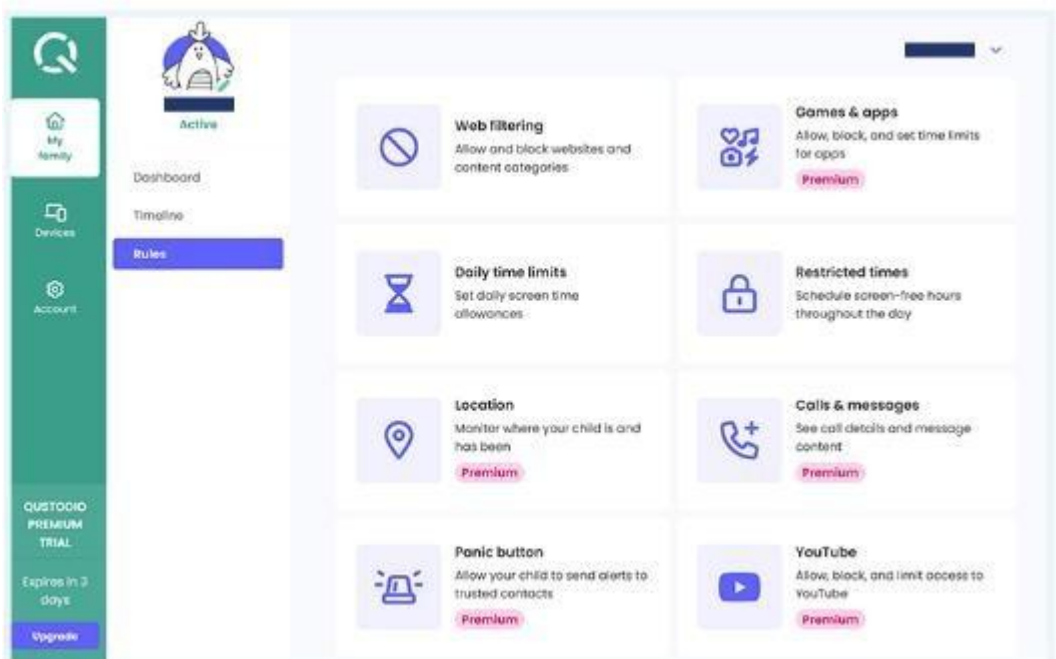


Figure 2.7 Qustodio Features

Source: (Qustodio, n.d.)

2.5.3 Net Nanny

Net Nanny is another example of a commercial parental control solution known for its content filtering capabilities. Net Nanny uses artificial intelligence to monitor and block inappropriate content in real-time, even on HTTPS websites. It provides parents with alerts, screen time reports, and control over device usage schedules. Net Nanny's strength lies in its advanced web filtering engine, which categorizes and evaluates online content dynamically. However, the system focuses more on content safety than on time enforcement or network blocking. It does not provide a network-layer lockout function upon reaching a screen time limit, nor does it support real-time remote commands from a dedicated dashboard using services like Firebase. This makes it less relevant for users who prioritize enforcement over content filtering.

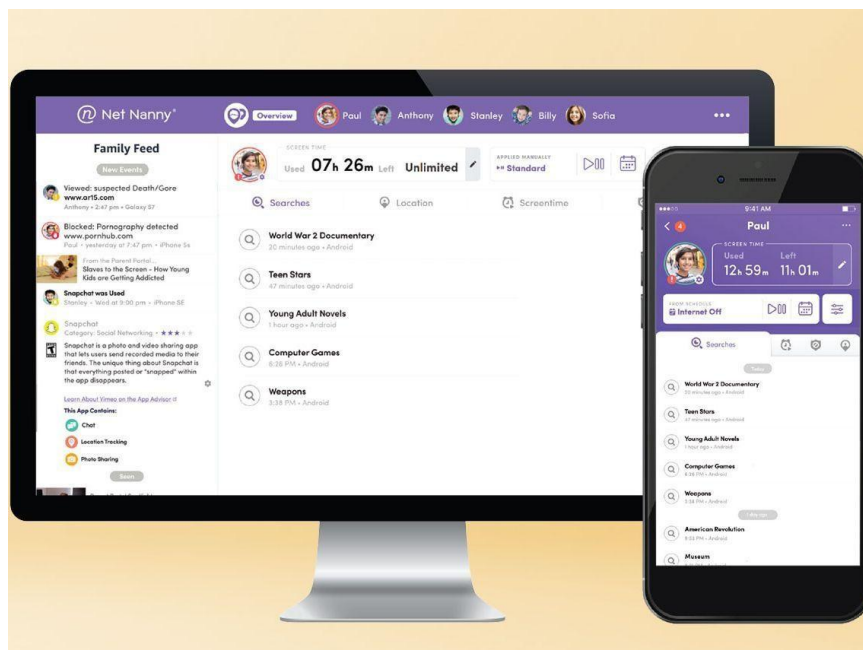


Figure 2.8 Net Nanny Dashboard

Source: (Google, n.d.)

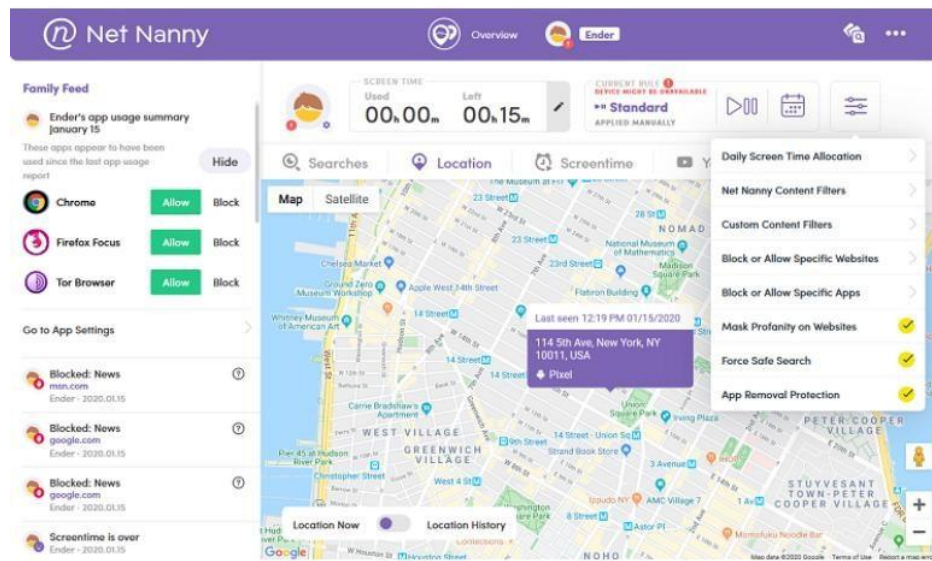


Figure 2.9 Net Nanny Homepage

Source: (Google, n.d.)

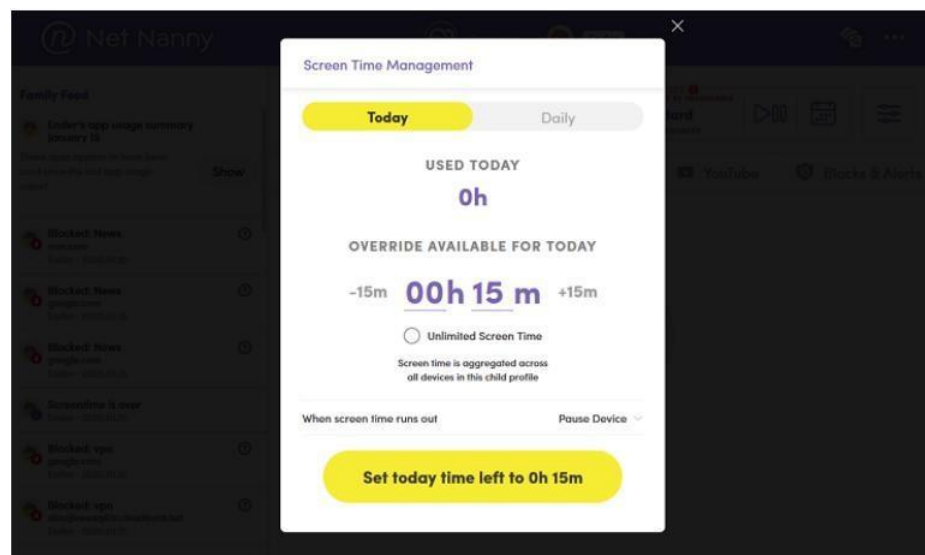


Figure 2.10 Net Nanny Screen Time Management

Source: (Google, n.d.)



Figure 2.11 Net Nanny Mobile View

Source: (Google, n.d.)

2.5.4 Apple Screen Time

Apple Screen Time, built into iOS devices, provides usage reports, app limits, downtime scheduling, and content restrictions. It allows parents to configure settings across family devices using iCloud. While the interface is intuitive and seamlessly integrated into the OS, the system suffers from key limitations: it is exclusive to the Apple ecosystem, lacks cross-platform compatibility, and offers limited enforcement depth. It does not include VPN-layer blocking, and users can manipulate device time or use third-party VPNs to bypass limitations. The proposed Android-based system in this project is designed to address such issues by enforcing restrictions at a lower system level and integrating Firebase for cross-platform command and control.



Figure 2.12 Apple Screenshot Dashboard

Page Source: (Apple, n.d.)



Figure 2.13 Apple Screenshot Summary

Page Source: (Apple, n.d.)

2.5.5 Comparison of Systems

Feature	Google Family Link	Qustodio	Net Nanny	Apple Screen Time
Platform Compatibility	Android, Chromebook	Android, iOS, Windows, macOS	Android, iOS, Windows, macOS	iOS, macOS only
Screen Time Tracking	Basic	Detailed	Detailed	Basic
App Blocking	App-based	App-based	App and web-based	Per-app limits
Web Filtering	Limited	Advanced	Real-time AI filtering	Content-based filtering
Network-Layer Enforcement	No	No	No	No
Remote Control (Admin Access)	Limited	Full	Full	iCloud-linked
Real-Time Sync	Periodic	Yes	Limited	Delay in updates
Tamper Resistance	Easy to bypass	Moderate	Moderate	Clock trick bypass possible
Cross-Platform Admin Dashboard	Web	Web and Mobile	Web	iOS only
Uninstall Protection	No	Partial	Partial	No
Pricing Model	Free	Freemium / Paid Plans	Paid Subscription	Free (built-in)

Table 2.1 Comparison of Systems

2.5.6 Literature Review Tree

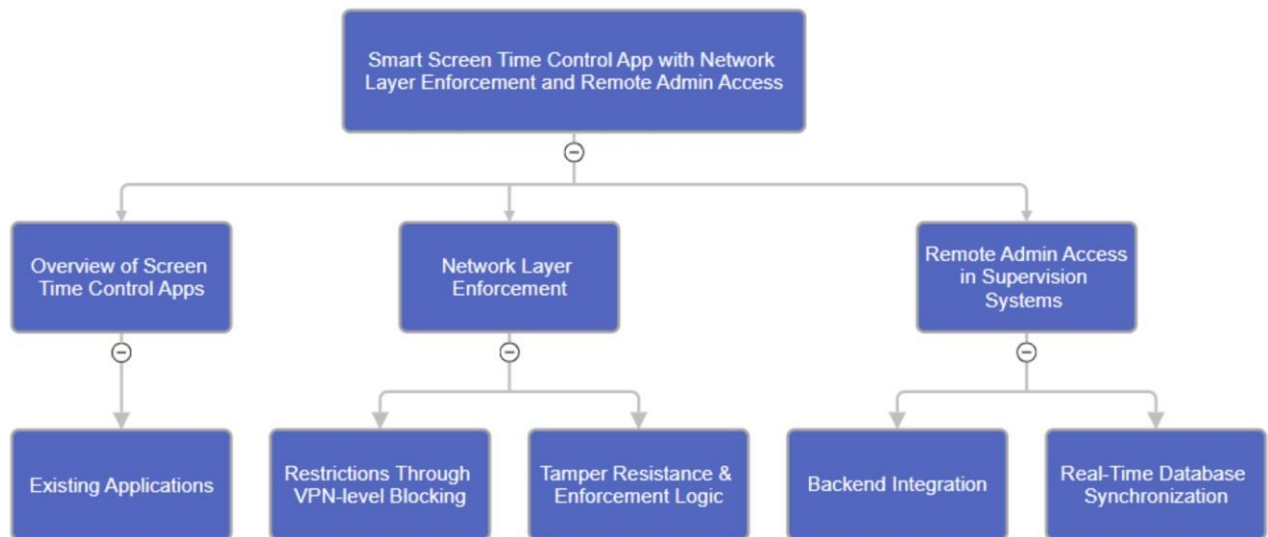


Figure 2.5 Literature Review Tree

CHAPTER 3

RESEARCH METHODOLOGY

This chapter explains the research approach and methodology implemented throughout the development of the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access. The methodology serves as a structured guideline to ensure the project follows a clear and systematic development path, allowing it to be completed within the given timeframe. This chapter also discusses the detailed process involved in planning, designing, developing, and testing the proposed mobile application. The app is designed to monitor screen time, enforce internet restrictions at the network layer using Android's VpnService, and enable real-time remote control through Firebase integration. Each phase of the methodology contributes to the secure, reliable, and functional delivery of the system to address current limitations in digital wellbeing and screen time management tools.

3.1 Project Framework

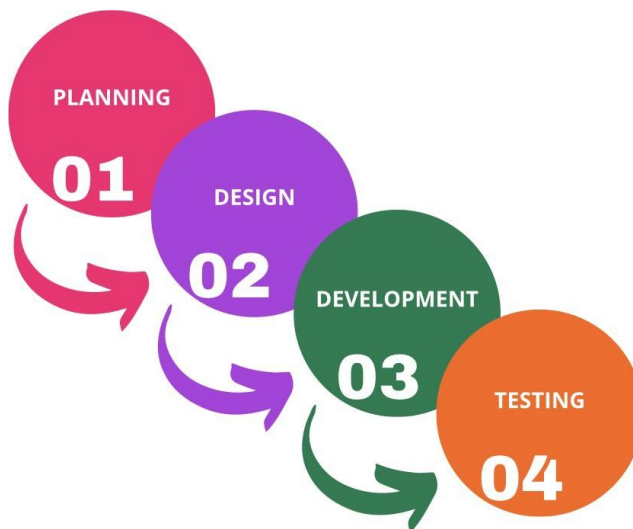


Figure 3.1 Waterfall Diagram

Figure 3.1 illustrates the Waterfall Model used to develop the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access to achieve the objectives outlined in Chapter 1. The Waterfall model used here follows four steps which are planning, design, development, and testing. Each phase is carried out in a linear and sequential manner. Each step must be finished before the next one starts, ensuring everything stays organized and on track. This structured approach ensures that the project remains organized and follows a clear timeline, while also supporting the development of a secure and tamper-resistant system with real-time monitoring and network-level enforcement.

STAGES	TASKS	DELIVERABLES
Planning	• Conduct research on existing screen time control apps and their limitations • Analyze user needs and problems from a digital wellbeing perspective. • Study the use of network-layer enforcement and Firebase in related projects.	• Background study on digital wellbeing and screen time issues. • Defined problem statement and proposed solution. • Clearly stated objectives, scope, and significance of the project. • Completed literature review.

Design	• Plan overall system architecture (user side and admin side) • Design UI layout for Android app and Firebase admin interface. • Identify suitable Android APIs (VpnService, UsageStatsManager) and Firebase features. • Draft use case, data flow, and system component diagrams.	• User interface mockups. • Firebase database structure design. • High-level system architecture diagram. • Defined roles for user and admin components.
Development	• Set up Android Studio development environment. • Develop screen time tracking functionality using UsageStatsManager. • Implement network-layer enforcement using VpnService. • Integrate Firebase for admin remote access and real-time sync. • Implement uninstall protection and background services. • Develop web-based admin dashboard	• Fully functional Android application. • Firebase Realtime Database with live command sync. • Working network-layer blocking functionality. • Admin access control and remote enforcement features.
Testing	• Conduct unit testing on screen timer, VPN blocking, and Firebase triggers. • Perform integration testing between user app and admin commands. • Test tamper resistance, such as forced shutdown or uninstall attempts. • Evaluate system responsiveness, accuracy, and stability.	• Functional and integration test results. • Finalized, stable mobile app with tested features. • Demonstrated system behavior under multiple use scenarios. • Bug reports and refinements documented.

Table 3.1 Project Framework

3.1.1 Planning Phase

The first phase, known as the planning phase, is essential as it lays the foundation for the entire development process. During this phase, initial research and documentation were carried out before any technical implementation began. The planning phase started with gathering information on existing screen time control applications and their limitations, focusing on issues related to enforcement, bypassing, and lack of real-time admin access. The problem statement was carefully analyzed to identify the key challenges that this project aims to address, particularly in the areas of digital wellbeing and mobile usage control. The project objectives, scope, and significance were clearly defined based on these findings. Additionally, previous research works and academic articles related to network-layer enforcement, parental control systems, and Firebase integration were reviewed to support the project direction and provide a solid background for system development.

3.1.2 Design Phase

The second phase, which is the design phase, is a crucial stage that follows the planning phase and precedes the development stage. The main objective of this phase is to transform all the requirements gathered during the planning stage into a clear, structured system design for the Smart Screen Time Control App. This phase outlines the technical and architectural criteria of the system, including:

- i. Selection of Android system components such as VpnService and UsageStatsManager.
- ii. Firebase service integration for real-time admin control.
- iii. User interface (UI) design for both the mobile app and the admin dashboard.
- iv. Logic around system behavior to enforce screen time limits and initiate network restrictions.

Furthermore, during this phase, system design documents, including use case diagrams, data flow diagrams, and component diagrams, which provide an overview of how the system will work, including the interaction with users and administrators, are compiled. Currently, flowcharts and mockups are made to visualize data flow and define the core features of app. This phase of design is essential to define the coherent framework and to prepare the system for an easy implementation in the next phase.

3.1.3 Development Phase

The coding and implementation is the third phase of the system mobile application development process where we write code and build the system based on the design specification. For this stage, all the software and hardware requirement were addressed such as android development environment, firebase integration, and hardware compatibility. Noteworthy libraries and Android APIs such as VpnService, UsageStatsManager, and Firebase SDKs were chosen to power key features such as screen time tracking, in-network enforcement, and real-time syncing of data. The app was written in Kotlin and built with Android Studio. Secure remote control by administrators was implemented using Firebase Realtime Database and Authentication. Tamper-Resistant (Background) Services Other tamper-resistances and internet-blocking logic also have been implemented and incorporated as part of reliable enforcement of the screen-time limits. Such a development methodology is described such that the system operates as user requirements dictate and in accordance with the objectives that were identified in the prior planning and design stages.

3.1.4 Testing Phase

Testing is an important phase in the software development life cycle (SDLC) in which the Smart Screen Time Control App is tested extensively for its proper functionality, performance, and robustness. This phase consists of a set of tests that serve to corroborate that the system is behaving as anticipated in differing

conditions. Crucial functionality, such as screen time tracking, VPN based internet blocking, Firebase command synchronization, and uninstall protection are all independently verified using unit tests. Integration tests are then carried out to ensure a smooth connection between the user's mobile device and the admin dashboard. Performance of real-time updates for actual data is tested to ensure that the admin commands are performed in time and with accuracy. Tamper protection is further verified by watermarking all data saved and posting to a simulated disable/uninstall routine. In general, this step makes it possible to catch and fix any bugs or problems before the app is deployed, so your app ensures that a safe and reliable user experience is possible.

3.2 Proposed Design

This chapter discusses the design of the Smart Screen Time Control App that integrates Android system services and Firebase for a secure, enforceable, and remotely manageable system. The architecture consists of three essential parts user mobile side application, admin side dashboard and communication infrastructure on backend. In sum, they combine forces to provide instantaneous command, dependable closure, and smooth action.

3.2.1 System Architecture Overview

The system is comprised of two main parts, a user-side application on the mobile phone and an admin-side interface through Firebase. These two work together enabling the remote real-time monitoring and control and the seamless synchronized data. Each subsystem cooperates to result in a responsive, reliable along with tamper resistant screen-time management technique.

i) User Module

User Module is a mobile app of which is downloaded to the user's child and student Android phone. This app is used to limit the time children spend with cell phone and limit them from using the device for more than a time length that is set. It will measure how long the screen has been on and what apps a user is running, and Android's UsageStatsManager API is used to do so. When the screen time limit is up, then the app support network-level blocking and leverages the VpnService API Android in order to block the internet on the device. That means people will not be able to circumvent restrictions by switching to another app or browser, all traffic will be blocked at the system level. The User Module connect into the Firebase Realtime Database to listen for control from the admin and log screentime data. Firebase Authentication will be used to authenticate users securely and also keep data safe and protected. To prevent attempts to offload the application, it also includes uninstall protection using Android's Device Admin

and services that provide the protection from user intervention, even after the device reboot. The app's User Interface could also feature visual cues or flashing indicators when the user is about to reach his or her screen time limit, thus prompting awareness and responsible use.

ii) Admin Module

Admin Module is a Web-Based dashboard which allows parents, teachers or administrators to monitor device usage and apply restrictions from anywhere. Admins log in securely via Firebase authentication so that only trusted individuals can access and administer the configured devices. The dashboard displays usage data in near-real-time by fetching screen time statistics from the Firebase Realtime Database, meaning administrators can check how much time the device has been used for, and whether any limits have been exceeded. From that dashboard, they can issue real-time commands like locking down internet access, giving extra time or temporarily unlocking the device. These commands are immediately pushed to the user's device via Firebase, so nothing lags or stutters. Multiple device management (when using the admin module) is also super handy when you are dealing with kids or school settings. Admins can customize rules such as screen time, daily limits, what happens when time is up, show a warning or immediately cut off internet access.

Combined, it is a secure, cohesive solution built on top of Firebase for data and authentication around which to build your app. But this configuration not only makes it difficult to tamper with screen time rules, it also gives parents and teachers real-time control so that they can intervene and make sure next-generation humans form whiz-bang patterns with digital media. By keeping the user role and admin role separate, and by tying everything together with a strong backend, the system remains easy to use, yet features great enforcement and user-privacy protection.

Role	Functionality	Description	Software/Tool Used
User	Screen Time Monitoring	Tracks screen usage duration and app activity.	UsageStatsManager API
User	Internet Blocking after Limit	Disables internet access when time limit is exceeded.	VpnService API
User	Data Sync to Cloud	Sends real-time data to Firebase for admin review.	Firebase Realtime Database
User	Tamper Protection	Prevents uninstalling or disabling the app.	Device Admin, BootReceiver
User	Warning Notifications	Alerts user when screen time is almost used up.	Android Notification Manager
Admin	Real-Time Monitoring	Views screen time data remotely.	Firebase Realtime Database
Admin	Set Time Limits	Defines how long the user can use the device per day.	Firebase Admin Interface
Admin	Lock/Unlock Internet	Remotely blocks or restores internet access.	Firebase Realtime Trigger
Admin	User Management	Manages multiple user profiles or devices.	Web Admin Dashboard, Firebase Authentication
Admin	Extend or Reset Time	Adds or resets screen time remotely.	Firebase Command Handler

Table 3.2: User and Admin Functionalities with Tools Used

3.2.2 Use Case

The use case diagram illustrates a more detailed aspect of your system to give more clarity in relation to its interaction between the non-system users such as admins and the users to accomplish a particular task or activity. It emphasises the action flow from the user or admin's perspective rather than what system does. The very rough sketch above just gives a high level view of the system, and it allows us to visualise now the different roles are allowed to interact with the app. The use case

diagram shows how user mobile app and how admin remote dashboard works with the Smart Screen Time Control App. It reveals how users are tracked for screens time, how internet filters are enforced and how admins can intervene from a distance to manage and control access. The architecture and interaction flow of the use case diagram is presented in Figure 3.2.

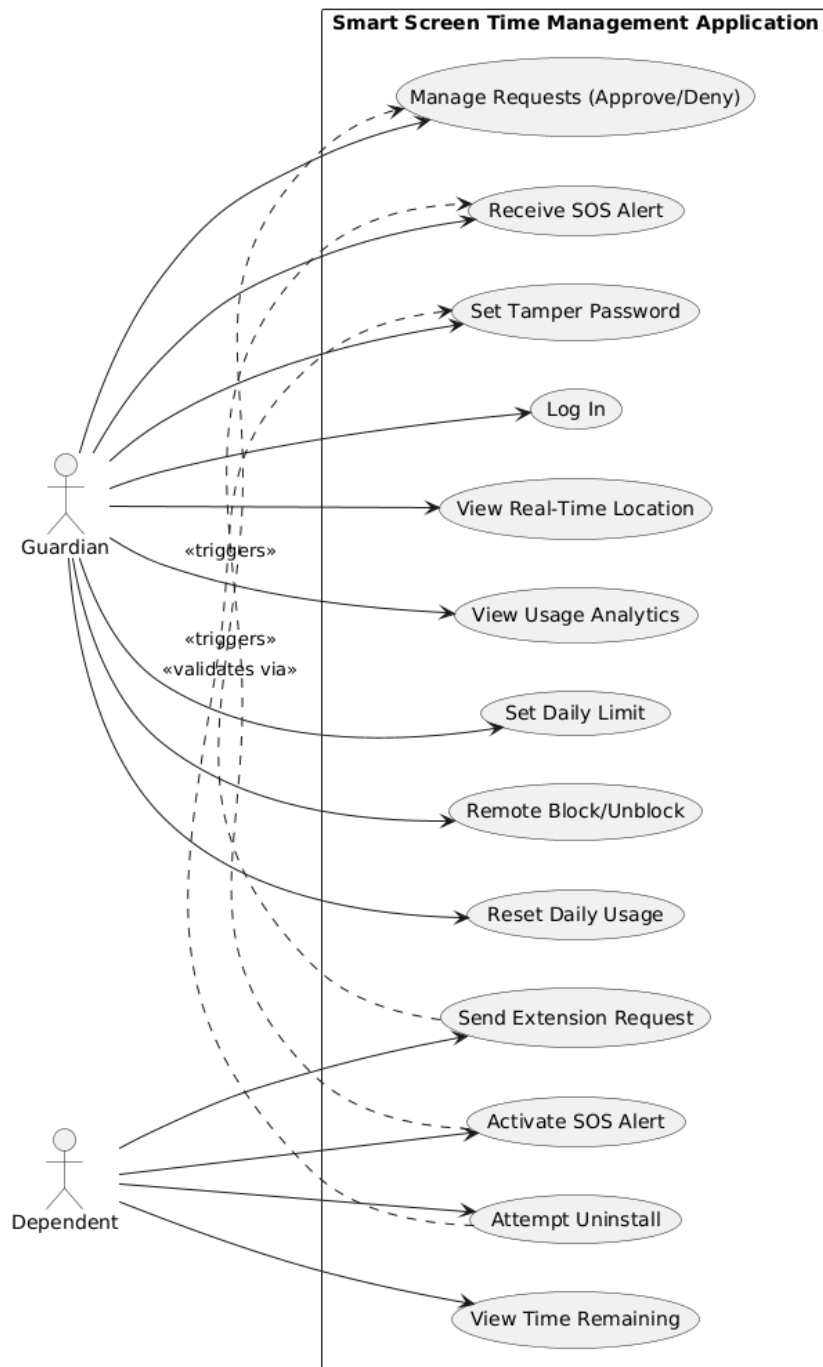


Figure 3.2 Use Case Diagram

3.2.3 Conceptual Diagram

The conceptual diagram is a visual representation of the major components of the system and illustrates how they are connected and how data flows among them, at a high level. It provides an understandable overview of how it all fits together without getting bogged down in the particulars. For this specific project, the diagram outlines the flow of how the user's mobile app, admin dashboard, and the Firebase backend will interact. It details how the screen time data is captured from the user's device and synced to the cloud, and how the admin can supervise and take action on the device, or remotely control the device with real-time commands. The diagram makes it easy to see how monitoring and enforcement are built into the system's overall design. The conceptual diagram of the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access is illustrated in Figure 3.3.

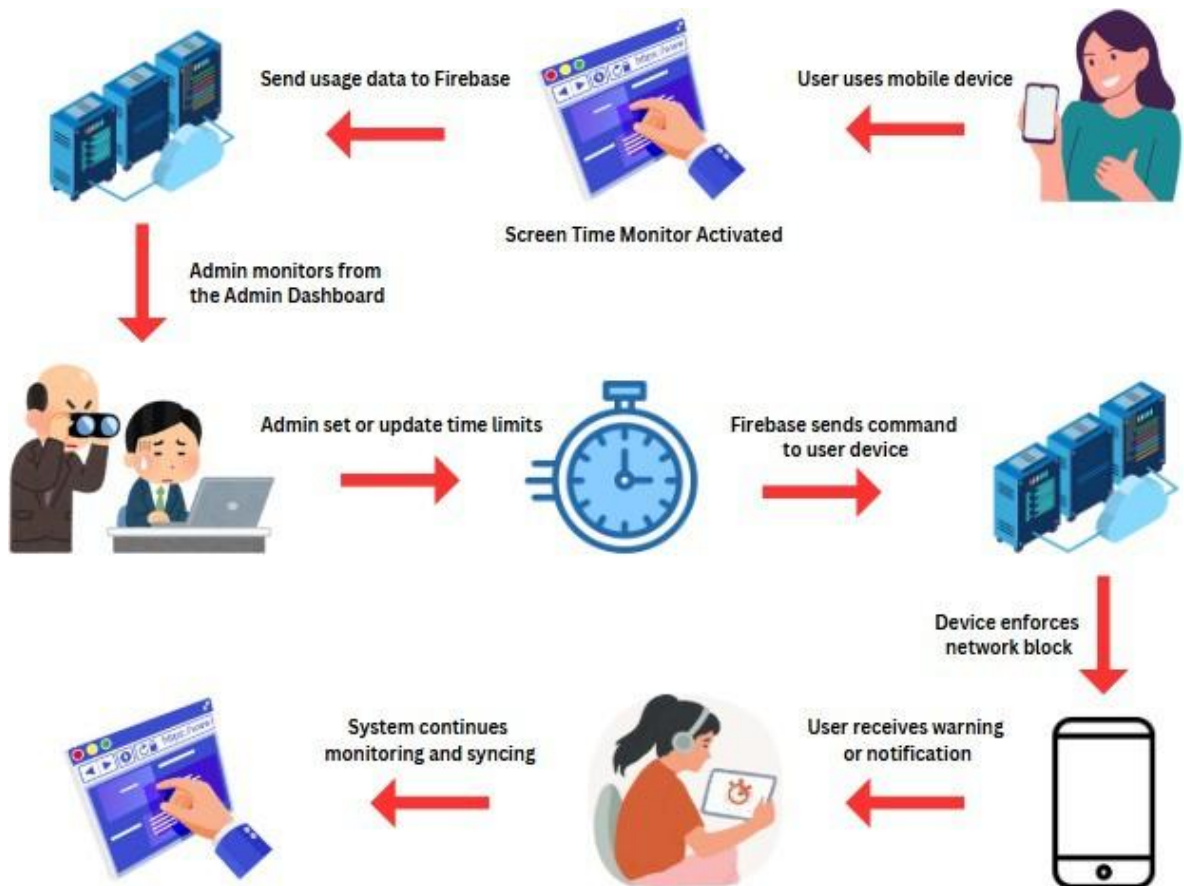


Figure 3.3 Conceptual diagram

3.2.4 Physical Diagram

A physical diagram is the visual representation that provides a more precise and fixed view of how the system really is deployed. It explains the deploy node, where software and hardware are placed and connected to implement the project in the practical realization of the project. This diagram let us observe the architecture of how the Smart Screen Time Control App is incorporated in this project, and how the mobile app, Firebase backend, and admin dashboard communicate with each other in real-time. It illustrates the path of screen-time data from the user's device to the database, and the path of control commands from the admin dashboard to the user's phone. It also contains the enforcement code, using Android's VpnService to restrict network access. The illustration of the system and its components is in Figure 3.4.

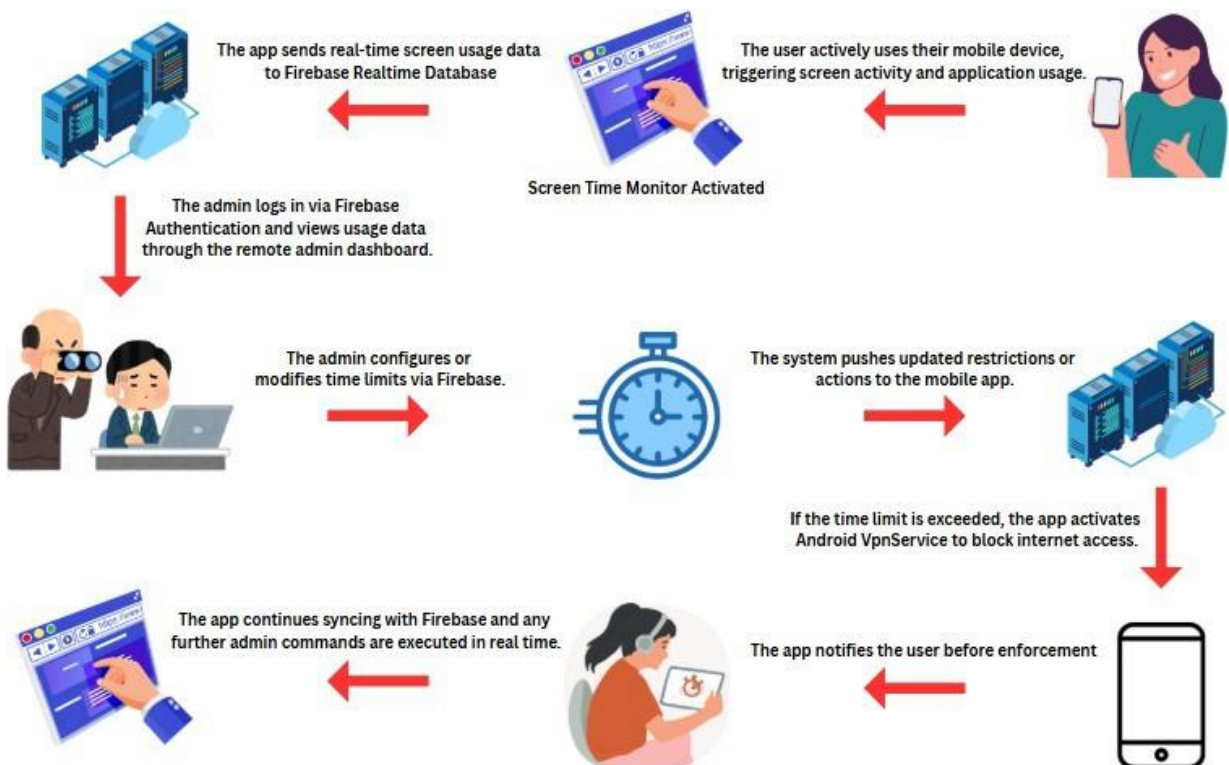


Figure 3.4 Physical diagram

3.2.5 Flowchart

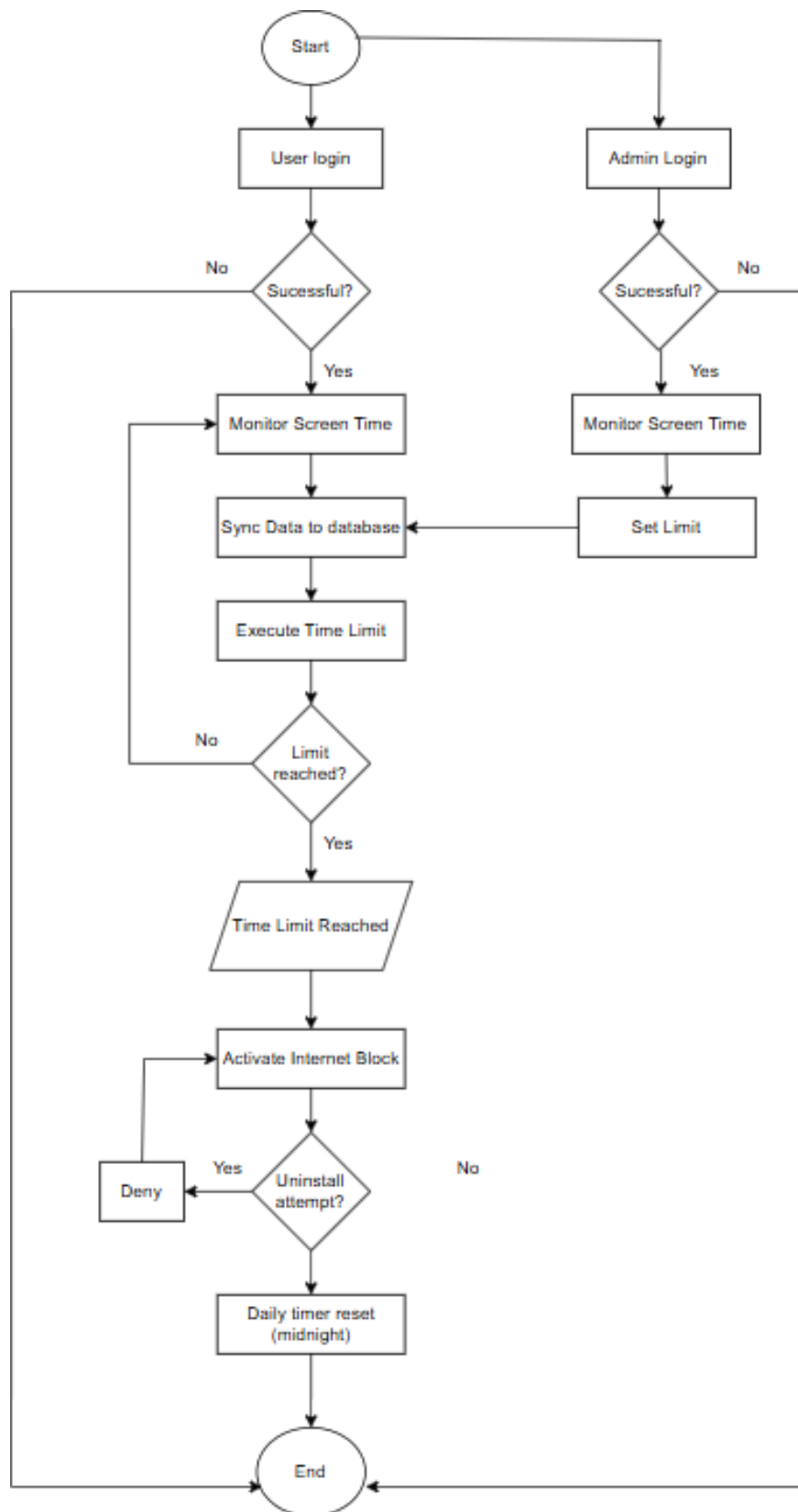


Figure 3.5 Flowchart

According to Figure 3.5, the flowchart illustrates the entire process of the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access. The process begins with either the user or admin logging into the system. Once authenticated successfully, the user module proceeds to monitor the device's screen time activity using system APIs. Simultaneously, the admin can monitor usage and set time limits from the remote dashboard. The user's screen time data is constantly synced with the Firebase Realtime Database. When the system detects that the screen time limit has been reached, it automatically triggers an internet block using the VPN-based enforcement method.

After that, the app checks for any intents to remove or meddle with it. The system will simply block anything suspicious and continue running normally. With or without tampering, the system renews its timer daily at midnight and has a clean slate for the new day. This flowchart demonstrates enforcement logic real-time communication between the modules, and tamper resistant behavior that comes together to define app operation.

3.3 Proposed Development

This project development is to address the implementations required to be performed building Smart Screen Time Control App with Network Layer Enforcement & Remote Admin Access.

3.3.1 Software Requirements

This section identifies the best software tools for developing the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access project. Table 3.3: The development tools and platforms that are going to be adopted in the whole development process of the mobile app and the admin dashboard. It is not just programming, it also includes backend services and UI work. These tools were hand-picked to assist in Android app development, facilitate real-time data syncing, and the secure communication between user and admin portions.

Description	Software	Function / Features
Development Environment	Android Studio	Integrated Development Environment (IDE) for building and testing Android applications.
Programming Language Support	Kotlin	Official language for Android development; used to build UI, services, and logic.
Cloud Backend Services	Firebase Realtime Database	Real-time data storage and synchronization between user app and admin dashboard.
User Authentication System	Firebase Authentication	Enables secure login for both user and admin; supports role-based access.
Network Enforcement	Android VpnService API	Allows the app to intercept and control internet traffic to enforce screen time limits.
Screen Time Monitoring API	UsageStatsManager API	Tracks screen activity, app usage time, and session history.
UI/UX Design Tool	Figma	Used to design user interface prototypes and plan screen layouts.
Version Control System	Git + GitHub	Manages source code changes and backups during development.
Admin Interface (Web Dashboard)	Firebase Hosting / Web browser	Allows admin to access dashboard for monitoring and control.

Table 3.3: Software Requirements

3.3.2 Hardware Requirements

To develop the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access, we need the correct hardware to use for developing, testing, refining, and finally deploying the system. A laptop is needed to write code, design the interface and integrate the backend, and a phone is needed to test how the app behaves, and obscures, enforcement capabilities. Table 3.4 show the hardware used in this project development including the Android device for testing use.

Hardware	Specification
Laptop	• Windows 11, Intel Core i7 Processor, 16GB RAM, NVIDIA RTX 3050 Ti GPU • Used for development, testing, and debugging in Android Studio.
Mobile Phone	• Android 15, Qualcomm Snapdragon 8 Gen 2, 12GB RAM • Used for running and testing the user module of the mobile app.

Table 3.4: Hardware Requirements

3.3.3 Libraries

The Smart Screen Time Control App with Network Enforcement and Remote Admin Access is developed in Android Studio with the help of many Android and Firebase libraries. Such libraries are instrumental for some key features like screen time tracking, internet blocking, user authentication, and cloud syncing. Their combination makes possible the operating of the system in real time, and the ability for remote control. The Firebase Realtime Database stores and syncs screen usage data and admin commands between the user's device and the admin dashboard and Firebase Authentication is leveraged for secure logins and access control for both user and admin roles.

Other fundamental libraries are Android's VpnService API to create local VPN interface that rejects internet connection at no screen time limits and Android's UsageStatsManager API to monitor total screen on time usage and particular app usage and log data. FCM (Firebase Cloud Messaging) may be added to notify and alert the user's device from the admin dashboard. These set of tools were specifically chosen to produce an efficient, secure and tamper-resistant mobile app which would satisfy all the goals of the project. Every library is developed and manually tested with the latest Android SDK versions to ensure everything is working in harmony with the latest models of android devices.

3.3.4 Android Mobile Application

An Android-based mobile application will be built to make the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access project. This App, developed using Android Studio, is the user end part of the system which integrates real-time monitoring, network level enforcement, and cloud syncing. It monitors screen time via the UsageStatsManager API for Android, providing insights on screen activity and app usage over time. Should the user exceed the set screen time, the app intervenes with restrictions by using Android's VpnService API to prevent access to the internet at the network layer for the entire device.

The app communicates with the admin using Firebase Realtime Database, so if the user locks the app or wants an extension of screen time, all data is synced in real-time with Firebase Realtime Database. It also alerts the user when time is nearly up or if access has been limited. It features tamper-resistance capabilities designed to detect when a user tries to force stop or uninstall the app, as well as a mechanism for automatically restarting background services to make sure everything keeps running. In general, the mobile app is the center of the enforcement system calling for, monitoring, restriction and communication is available any time in a safe and user-friendly manner.

3.3.5 Testing

There are multiple steps in testing the Smart Screen Time Control App with Network Layer Enforcement and Remote Admin Access to ensure the system functions properly and dependably. Testing begins with investigation of client-side features, primarily screen-time tracking, to respect that the application indeed measures screen time by using Android's UsageStatsManager API. Then, the internet restriction mechanism is validated to guarantee the VpnService API is properly invoked at the time the limitation is reached, and blocks the internet access, while not disrupting other background activities.

Apart from the testing of the user-side features, the admin features are also tested by the remote dashboard. This involves verifying the sync live between the admin interface along with the mobile application together with the Firebase Realtime Database. Admin actions, such as locking, unlocking and extending screen time, are tested to ensure they are delivered and executed on the user's device properly. It also checks if user tries to uninstall or force stop app, it will get detected and will automatically restart its services. The last test confirms day reset and make sure screen is reset on midnight. It is via this exhaustive testing process we're able to verify that the system does secure enforcement and reliable syncing, and offer users and admins an easy experience.

CHAPTER 4

RESULTS AND FINDINGS

The chapter presents the entire findings of the design, development, and testing of the Smart Screen Time Control App. The essence of this project was to develop a powerful system that could allow to enforce a limit on screen-time by deploying network-layer blocking on one hand, and offer a wide range of remote administrative features on the other hand. The system consists of Android mobile application (User Module) that uses VpnService provided by Android to enforce and a tamper-resistant interface and the Web-based Admin Dashboard (Admin Module) to monitor and control in real-time. The findings confirm that the system can impose restrictions, monitor the location, control the digital wellbeing by locking the phone to Prayer Time, and offer the fundamental safety features such as SOS messages.

4.1 Proposed Development

The solution is designed according to a dual-interface architecture that allows secure and transparent communication between the guardian and the target device that implies that the mobile end promptly enrolls each of the administrative commands. Such an architecture is built on some notion of live data streaming whereby the administrator maintains a fixed surveillance of the digital activity, geographical location and device status of the user, without necessarily having physical access to the smart phone. The list below gives a detailed analysis of the individual interface pieces as well as the logic behind the major functionalities of the system itself.

4.1.1 Admin View (Web Dashboard)

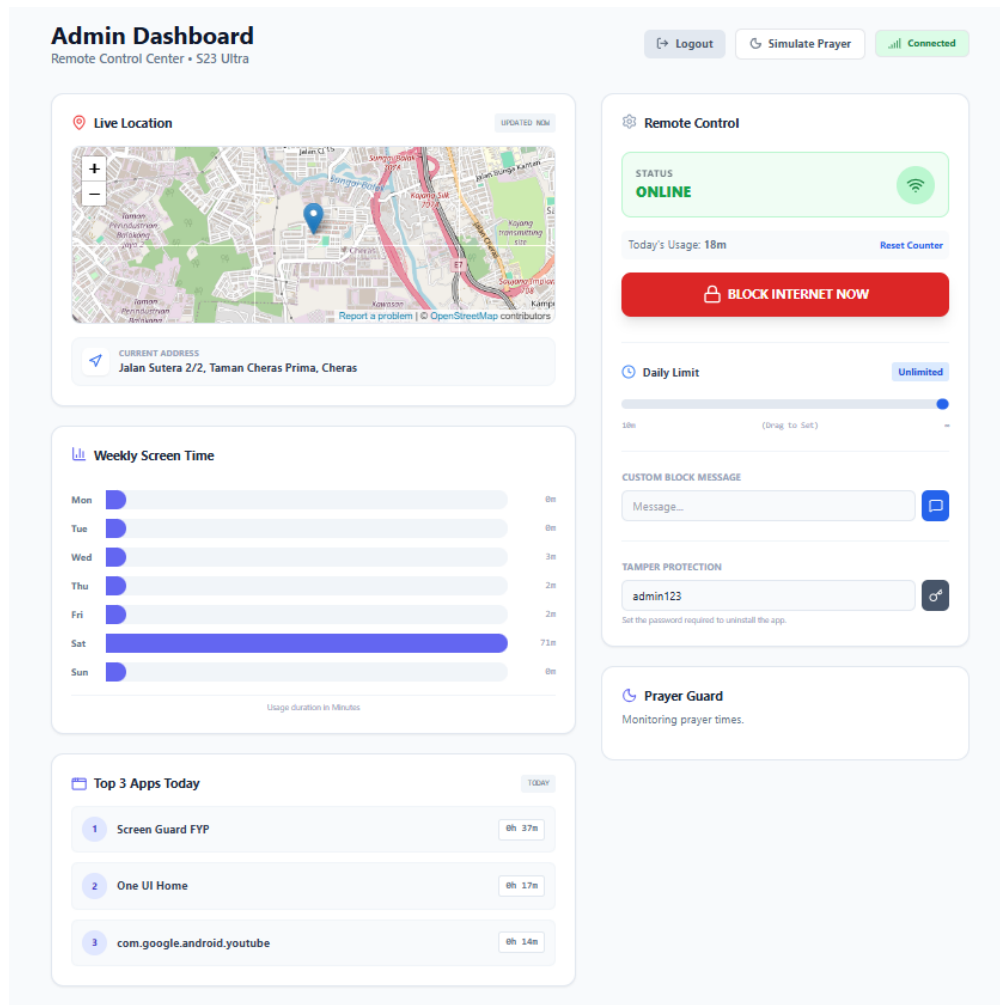


Figure 4.1 Admin Web Dashboard

The administrative dashboard is the central command hub, and it displays a complex user interface to facilitate the simple comprehension and depiction of data in an authorized and secured environment. After being logged-in to the system successfully by the guardian, they are shown a live map interface that will convert readable addresses of the exact GPS coordinates into the finer breakdown of weekly use pattern in minutes, and a horizontal bar chart. Moreover, there is a highly reactive control panel present on the dashboard by which the administrator can perform instant internet suspension,

change daily limits through a dynamic slider and a special option available on the control panel named Reset Daily Usage through which the administrator can programmatically reset the day counter to provide additional time where necessary.

To have the system run continuously, some sort of continuous feedback is available via the dashboard where connection status indicators and loading states are available to help avoid overlapping commands when updating the database. Through the use of the personalized message feature, the admin is able to issue instructions or warnings in a manner that is both delivered to the user on screen immediately a block is active and a more humanitarian approach to digital discipline is promoted. The authoritative control and granular usage analytics enable guardians to make decisions about what is optimal to the digital health of their child in the most efficient way possible.

4.1.2 User View (Mobile Application)

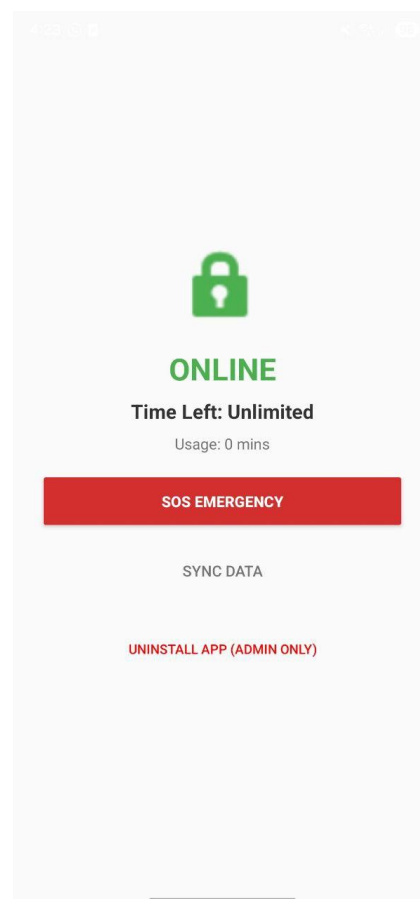


Figure 4.2 Mobile Application View

The mobile interface that is currently under implementation on the target device is so designed as to be informative but at the same time, restrictive enough to ensure that the user is on the alert of the current state and the usage window. Another equally critical feature of the general screen is the display of time left which has the ability to determine the difference between the limited time that has been imposed and the cumulative aggregate usage in real-time so that the user can readily feel as though their action has been restricted to control their behavior. Upon entering a blocked state (limit exhaustion, admin override or prayer time lock), the interface enters into high-visibility alert mode during which the administrator will show a custom message restricting all normal navigation by giving a priority to enforcement logic.

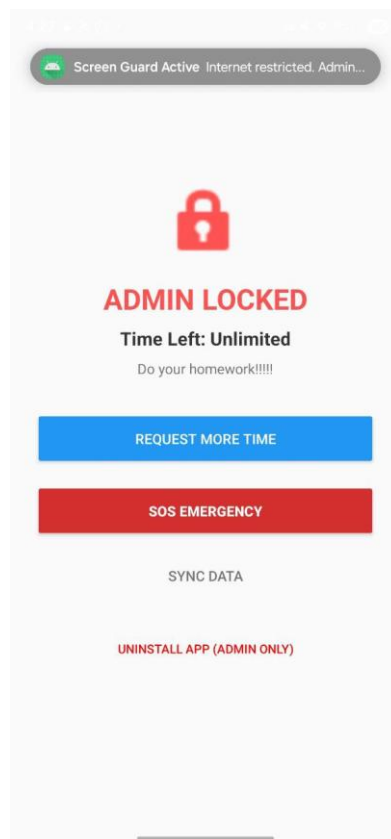


Figure 4.3 Admin Locked View

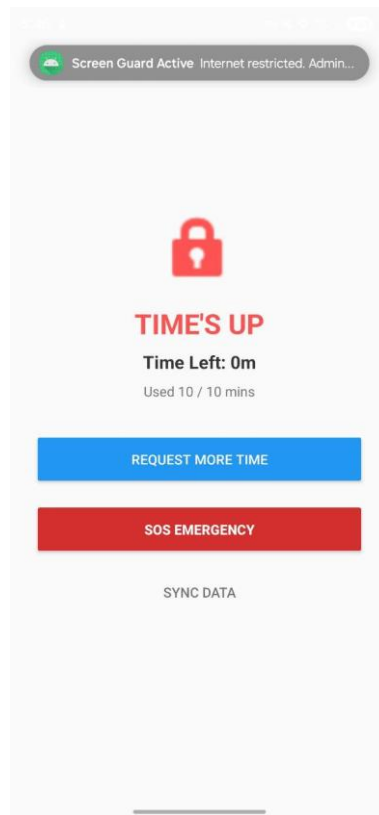


Figure 4.4 Time Limit Reached View

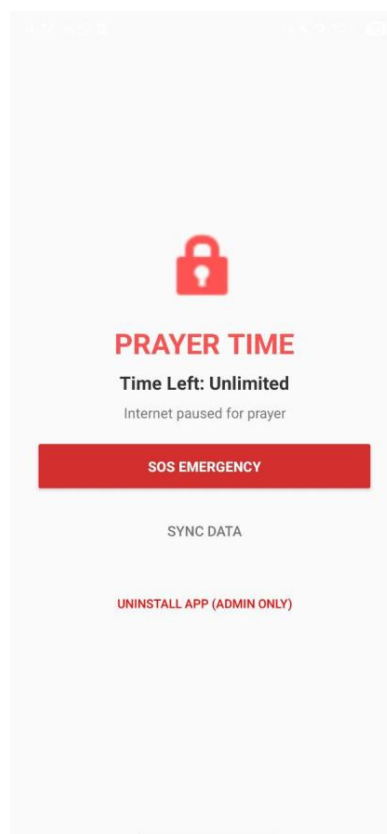


Figure 4.5 Prayer Time Lock View

In addition to restriction, we will provide a more than restriction feature in the form of a safety-critical SOS Emergency Button, which the user can always access and which will make the device a life saver in life-threatening situations regardless of any active screen time blocks. When this button is activated, the app ignores regular communication limitations and makes the device send accurately geolocation information to the Firebase database, which triggers the red banner warning on the administration dashboard to emit alerts and the guardian to be alerted via an automated email. It is also a move in the right direction of the design of these parental controls because it acknowledges that the user safety should be put first before the digital restriction, thus forming a more balanced and protective mechanism that can be used by the modern families.

4.2 Testing Results

A number of exhaustive tests have been performed on the Samsung Galaxy S23 Ultra with Android 14 to test functional accuracy, network reliability, and security robustness. Every test case was built to simulate real-world scenarios, for example moving between locations to test GPS tracking, or trying to circumvent internet blocks through web browsers or applications. We obtained consistent results suggesting that the system continues to communicate with the Firebase backend and the command execution delay is normally less than 2 s, thus the chosen architectural stack is well suited for real-time remote management.

4.2.1 Functional Testing

Functional testing verified that all the basic and secondary operations were fully functional as outlined by the project conditions (new reset commands applied, safety alerts issued). In the "Reset Daily Limit" test, the web dashboard successfully updates the bonus time node in the database with just a single click, interpreted directly by the mobile app to enable additional use without restarting the device. Similarly, the SOS alert system was tested by triggering the panic button under various network conditions, which always delivered the coordinates and triggered the emergency interface on the administrator's end, demonstrating the reliability of the safety-critical components. Table below will show all of the test case that had been done.

Test Case	Test Description	Expected Result	Actual Result	Status
1	GPS Tracking Accuracy	Admin dashboard updates map location when the device moves.	Dashboard showed updated address "Taman Cheras Prima" within 5 seconds of movement.	Pass
2	Remote Blocking Latency	Clicking "Block" on the web disables internet on the phone immediately.	Phone screen turned red and internet access stopped within 2 seconds.	Pass
3	Usage Data Synchronization	Phone usage data (app time) syncs to Admin chart.	Weekly usage chart updated on the dashboard reflecting correct usage minutes.	Pass
4	Prayer Time Enforcement	App auto-locks during scheduled prayer times.	App entered "Prayer Mode" automatically at the designated time.	Pass
5	Daily Limit Time Slider Functionality	Admin set daily limit slider to 20 minutes from the web dashboard and the time limit updated immediately.	The time limit updated immediately on the mobile app when admin adjust the daily limit slider.	Pass
6	Custom Block Message Sent	Message is sent when admin clicked on block internet.	Message received at the mobile app when admin blocked the internet.	Pass
7	Tamper Protection Security Test	Require password for app uninstallation via Device Admin.	Prompted for password blocked uninstall after incorrect attempts and alert the admin via Admin Dashboard	Pass
8	SOS Alert Test	Pressing SOS button sends location alert to the Admin Dashboard	Admin Dashboard showed red SOS banner received with coordinates.	Pass

Table 4.1: Test Cases

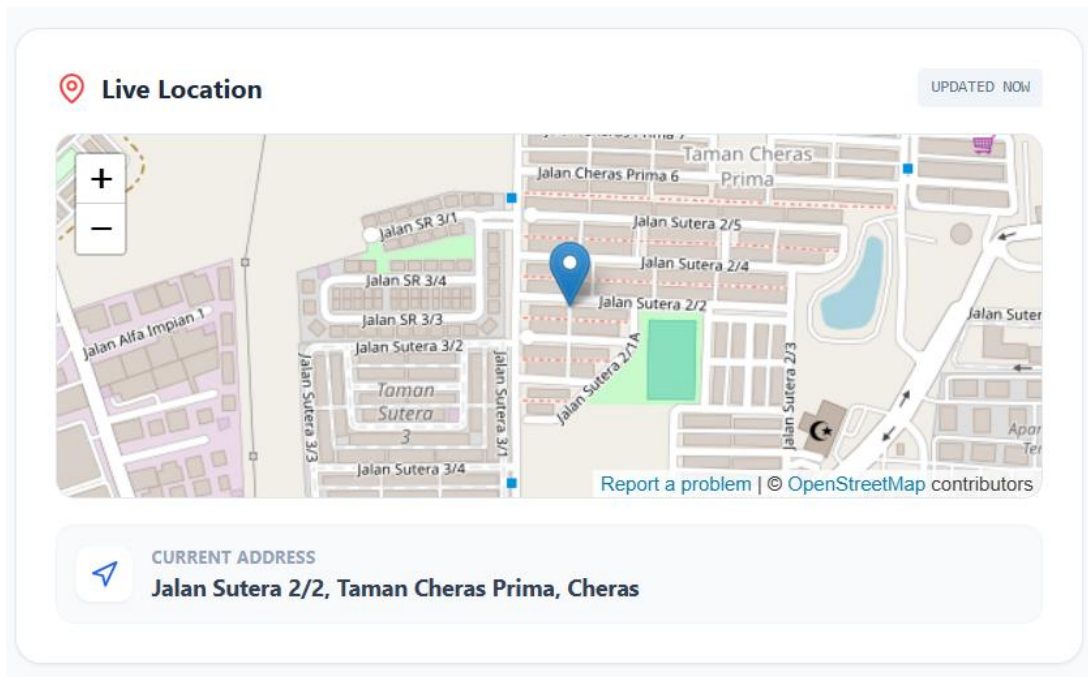


Figure 4.6 Test Case 1 Result

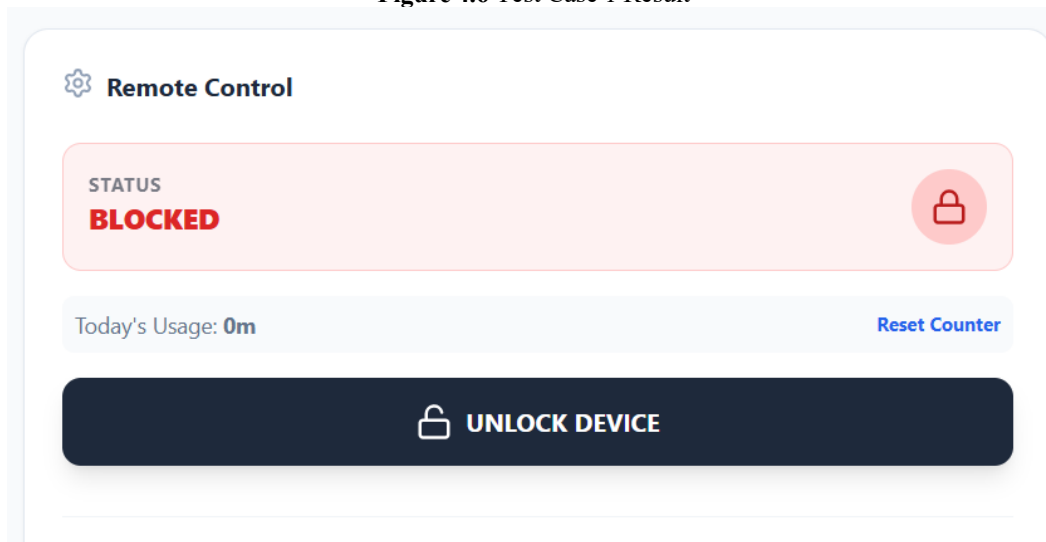


Figure 4.7 Test Case 2 Result Admin's View

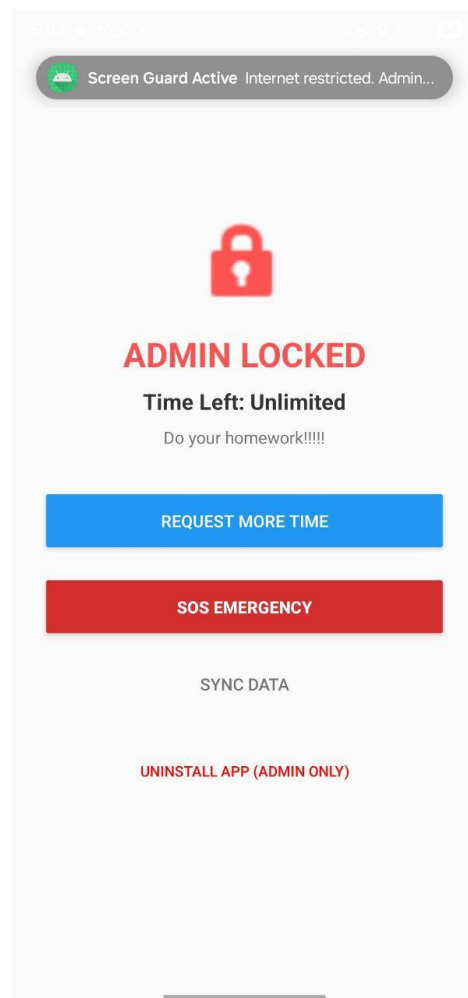


Figure 4.8 Test Case 2 Result User's View

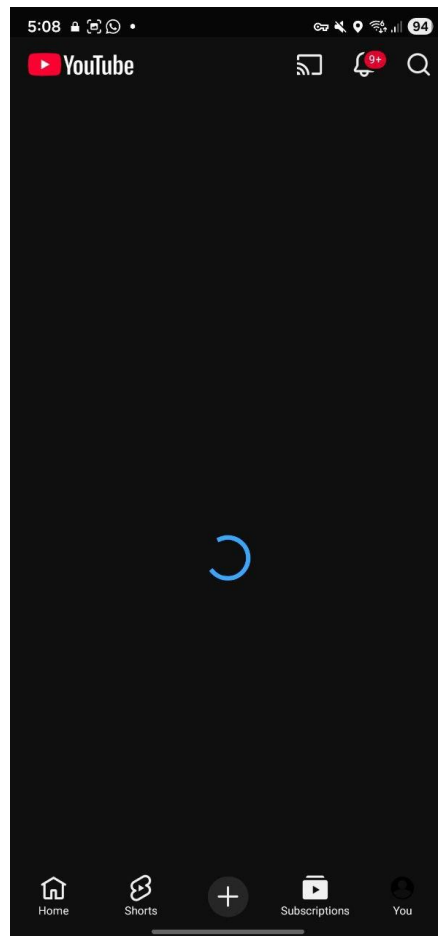


Figure 4.9 Test Case 2 Internet Is Blocked

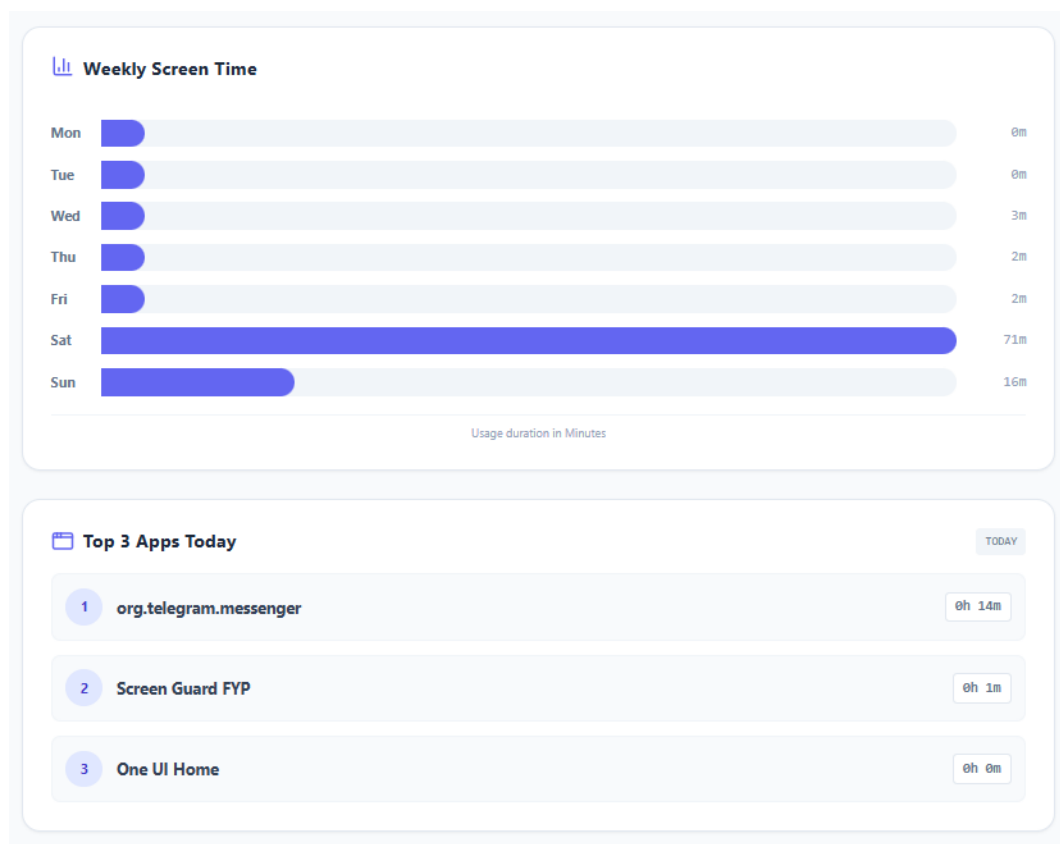


Figure 4.10 Test Case 3 Result Admin's View

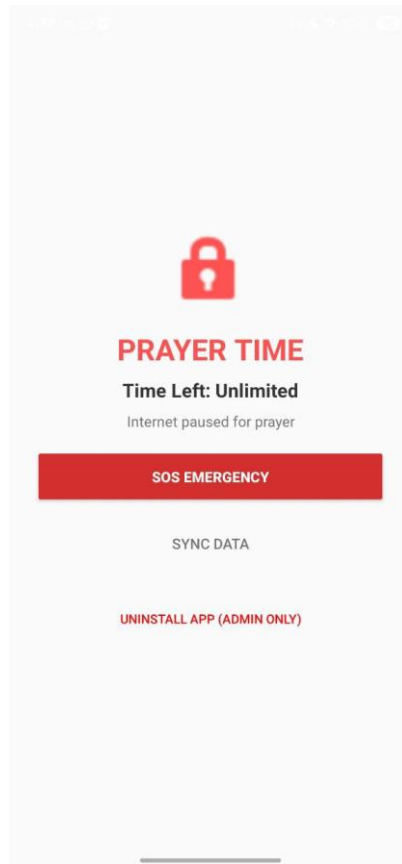


Figure 4.11 Test Case 4 Result

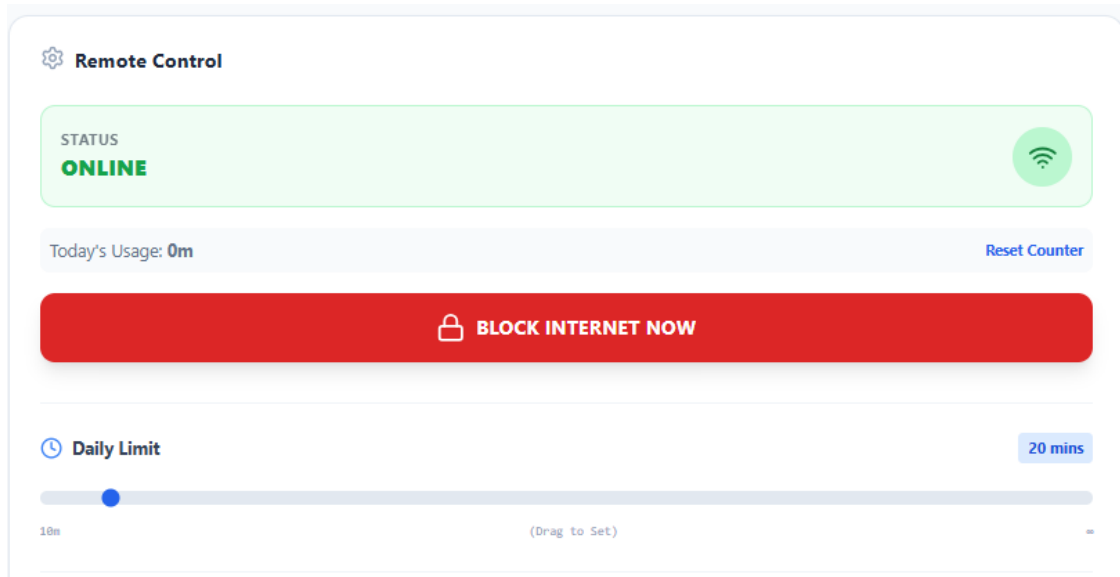


Figure 4.12 Test Case 5 Result Admin's View

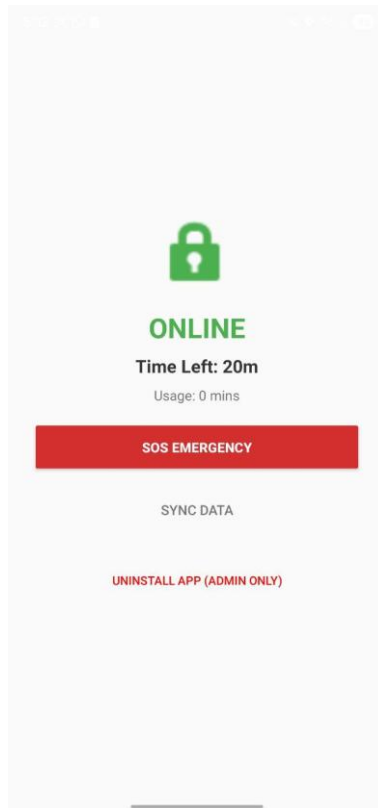


Figure 4.13 Test Case 5 Result User's View

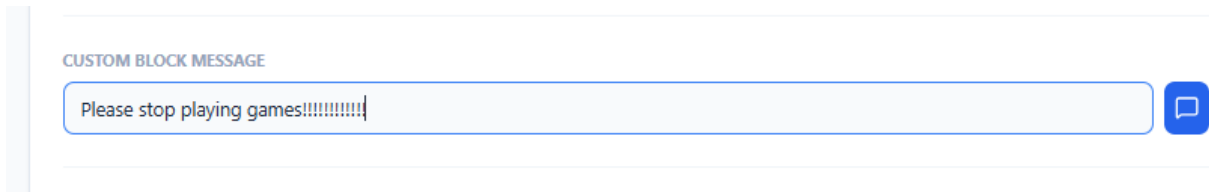


Figure 4.14 Test Case 6 Result Admin's View

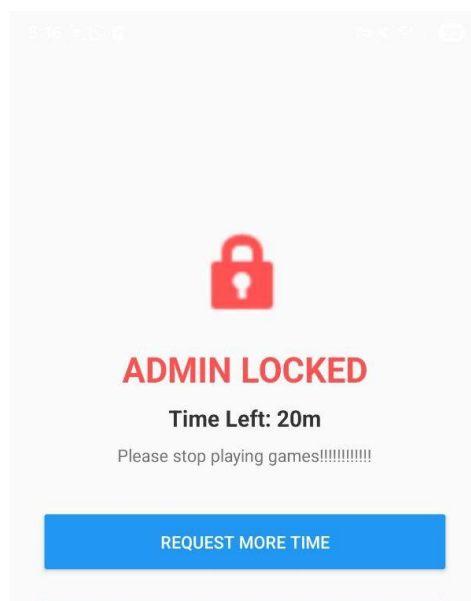


Figure 4.15 Test Case 6 Result User's View

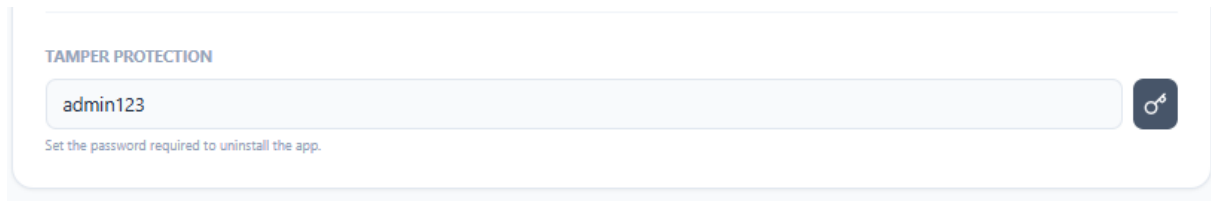


Figure 4.16 Test Case 7 Admin Setting Up Password for Uninstalling Purpose

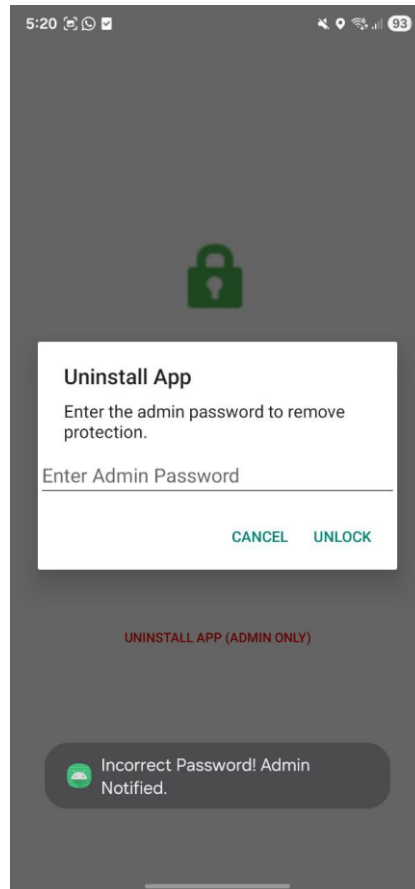


Figure 4.17 Test Case 7 User Entered Wrong Password

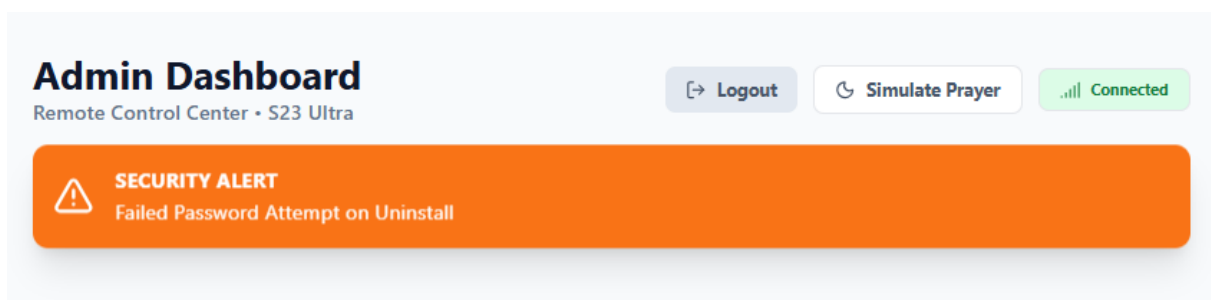


Figure 4.18 Test Case 7 Admin Alerted on the Admin Dashboard

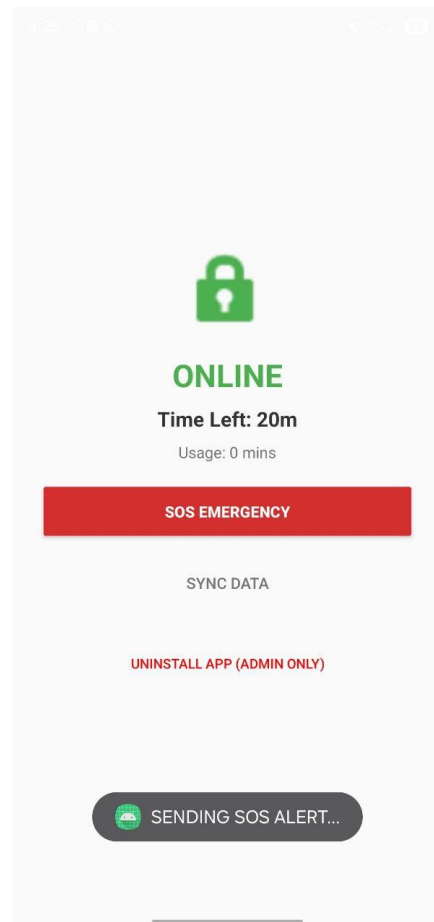


Figure 4.19 Test Case 8 User Pressed SOS Emergency Button

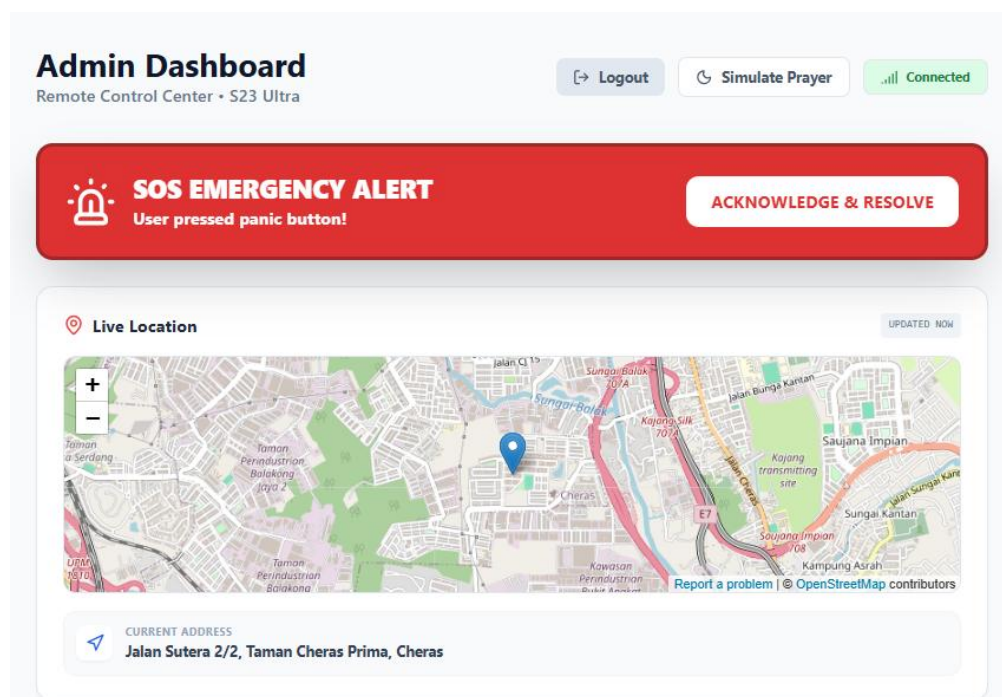


Figure 4.20 Test Case 8 Admin Received the Alert on the Admin Dashboard

4.2.2 Security and Tamper Resistance Testing

Security testing was conducted in a secure approach, which was well crafted, which was supposed to be the first test that could test whether the system was resilient against unauthorized attempts to disable or remove the application and would resolve what could be the biggest bug in the standard parental control software, where many tech-savvy users simply uninstalling the management tool will eliminate restrictions. The application was rooted at the Android Device Administrator API, which was used to access the OS's security layer, and disabling the typical features such as "Uninstall" and "Force Stop" in system settings menu for any user who is not an admin. To ensure that the application could be removed legitimately, while keeping the user's environment as secure as possible, a specific "Secure Uninstall Mode" was added to the mobile application interface whereby a unique administrator-defined password must be entered to relinquish the device's admin rights. Test results verify that any attempt to circumvent this protocol, be it through system settings or external file managers, was intercepted, ensuring that the network enforcement layer remains active and persistent in the face of persistent user attempts to regain unrestricted device access.

In addition to the physical prevention of uninstallation, testing also tested the extent to which the platform offered proactive security forewarning of issues via a new-generation smart "Tamper Alert" between the mobile device and web dashboard. In this evaluation, controlled failure scenarios were run in which intentionally entering a wrong password into the uninstall challenge was modeled as well the solution successfully detected unauthorized activity before calling the Firebase Realtime Database in a timely call, thus allowing the application to raise a high-priority message. The React-based admin dashboard later used this event-driven logic to trigger a high-visibility security banner and log every single precise timestamp of the attempt, a clear proof that the system is not only very reliable in resistance to removal, but it does know when to notify the guardian of defiance in real time. This layered security paradigm establishes a holistic deterrent, combining punitive technical enforcement along with an informative feedback loop and effectively completing the project objective of enabling a tamper-resistant ecosystem for the management of digital wellbeing.

4.3 Discussion

The results from this project prove convincingly that the use of network-layer enforcement through the Android VpnService API and system-level tamper resistance forms a control paradigm that is much more effective than the traditional approach of application-layer enforcement. Although the majority of commercial screen time applications depend on accessibility overlays or periodic activity monitoring which both of which are known to be circumvented by simply force-stopping background services or clearing out the application cache, the VPN-based architecture proposed in this work works at the network stack level. By encapsulating and selectively dropping data packets at the source, the device's connection to the internet is totally inoperative the moment a block occurs, rendering social media, streaming services, or alternative browsers virtually inaccessible. The move from "monitoring" to "hard enforcement" helps to remedy the core technical vulnerability identified in the problem statement and helps ensure that the digital perimeter is firm and unshakable regardless of the user's technical proficiency or attempts at circumvention.

In addition, the multi-dimensional functionalities, for example, SOS panic notification button and localized prayer-time guard, make this project a holistic digital wellbeing ecosystem and not a restriction tool. Discussion of these findings points to a fundamental change in parental control philosophy acceptance of the fact that a managed device must still be a safety lifeline for the user. With the SOS module eliminating all active internet blocks to dispatch emergency coordinates and send automated email alerts, the service offers parents a level of security that restricting apps often lack. Furthermore, the successful implementation of the prayer time simulation suggests that automated enforcement is able to be used to nurture good habits and spiritual discipline and that it may be at the heart of the future, and in fact point to adaptive digital wellbeing software to the cultural background and safety needs of the family unit, rather than offering a one size fits all restriction model.

4.4 Summary

This chapter has presented an in-depth description of the successful development, integration, and validation of the Smart Screen Time Control system, providing empirical evidence that a complete realization of the main project objectives has been achieved. From establishing a secure, authenticated admin login, and creating informative horizontal bar charts for usage analytics, to adding a strong password-protected uninstall mechanism and safety-critical SOS alerts, the system is a full-fledged technical solution for contemporary digital management. It has also demonstrated that the Kotlin-based Android application interacts with the React-based web dashboard in unison to deliver high quality and reliable data flow across the Firebase Realtime Database backend. The core functionality, including the remote daily limit reset and real-time GPS tracking was all tested thoroughly, ensuring both that the architectural design is scalable and technically sound.

The extensive testing data performed on a high-end Samsung Galaxy S23 Ultra operating under Android 14 additionally indicates that it is stable, responsive, and resilient against standard bypass methods and background service restrictions. This project showed that high-level administrative control can be achieved without sacrificing execution, performance, or system integrity, and by successfully leveraging foreground services and special use declarations to resolve the obstacles of modern Android security policies. The findings in this chapter, ultimately, strengthen the foundation for the study concluding that a network-layer enforcement approach, in tandem with real-time remote administration and proactive safety features, is one of the more feasible and reliable systems for effectively controlling digital addiction and safeguarding users in a connected world.

CHAPTER 5

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

The goal of this project was to create a strong, network-level screen time management system that addresses technical weaknesses in modern parental control apps effectively. This research has proven to be successful as it has built a complete digital wellbeing ecosystem a tamper-resistant Android application and a real-time, React-based web administrative dashboard. We have shown that a network-based enforcement solution based on Android VpnService not only works as a technical approach, but is much more powerful compared to the basic application-layer blockers commonly used that are vulnerable to simple bypass methods like force-stopping applications or manipulating system clocks. The system operates at the network stack level, ensuring that internet access is controlled at the source. In so doing it provides guardians with a dependable tool for fostering healthier digital habits, combating what has now become an increasing problem which is smartphone addiction.

The system exhibited robustness on numerous important performance metrics during the iterative development and testing stages, notably in its ability to synchronize administrative commands with negligible latency. This secure, authenticated admin login system allows only authorized guardians to access the admin control features, and the use of informative horizontal bar charts serves to bring data-driven insights to allow for informed intervention. In addition, the establishment of the "Time Remaining" countdown and remote daily limit reset feature gives authoritative control with transparency to enable a medium of communication, making it more flexible for screen time management. It is the synergy of the mobile client and the cloud-based backend that confirms the selected architectural framework as a high-performance solution for real-time remote device administration.

In addition to the restrictions offered by the system, the project makes a notable contribution to user safety using an integrated SOS Emergency Alert system. Because the managed device is still considered a lifeline to the end-user in a life-threatening situation, crucial location and automated email notification functionality will be able to deliver alerts even if a general internet

block is active. In addition to a robust tamper-resistance module which protects against unauthorized uninstallation with a secure password challenge, the system maintains a persistent and protective environment that remains active under all conditions. On the whole, this project contributes to a practical and scalable technical blueprint that will help the next generation of digital wellbeing tools, providing a link between simple monitoring apps and enterprise-grade management systems to provide parental peace of mind in an age of ever-growing connectivity.

5.2 Limitations

Even though main goals had already been achieved and the main project objectives were largely met, a number of technical issues and functional constraints were observed during the project development and testing stages. The most important difficulty is the exclusive dependence on the Android VpnService API for enforcing the network-layer rule, which, due to architectural restrictions within the Android operating system, stops the application from being used simultaneously with other VPNs. This means that if a user needs a separate VPN to access a corporate network or browse privately, the enforced screen time would necessarily be turned off, which leaves a gap available in scenarios where multi-layered network protections are necessary. This “one-active-VPN” policy is still one of the platform's core constraints, demanding a choice between external connectivity tools and internal screen time control.

Moreover, it is currently implemented in the Android ecosystem only, which in turn makes it less applicable to multi-platform households or organizations with mixed devices. For example iPhones, iPads, and Windows-based tablets making the product unsuitable for multi-platform environments. Inherently, the web-based administrative dashboard is cross-platform, but enforcement logic can only be enforced on the user front through certain Android system APIs that lack direct equivalents in the iOS environment without extensive architectural re-engineering. This lack of cross-platform compatibility means guardians cannot have a consistent management strategy across all the family devices and so the future versions of the software require a more platform-agnostic approach. Moreover, the system's dependence on continual Internet connectivity to synchronize commands in real-time and transmit SOS alerts also greatly limits its administrative efficiency in environments with sparse cellular coverage or no network access.

5.3 Recommendations for Future Work

To complement the successful prototype successfully built in the present work, we discovered technical weaknesses that we must overcome. The future development plan for the system should include an "App Whitelisting" or "Selective Traffic Filtering" functional aspect. Another option possible would be using something similar to the `addDisallowedApplication` method in the VPN builder, perhaps deep packet inspection, to allow an administrator to filter a list of educational or utility applications which remain active, even when a general internet block is enabled. As a result, the system would become a more refined productivity system for students and allow them to use essential digital support for academic work whilst still being protected from distractions through the larger web.

Platform expansion and technology integration, but in order to be able to realize cross-platform management the new version of iOS based on Apple's Network Extension framework needs to be made. Growing the system to encompass the Apple ecosystem would result in a more cohesive parental control experience across multiple device families in one household. A further improvement that can be achieved is a potential added support of AI-driven analytics and machine learning models to offer responsible guardians to obtain more nuanced, predictive insights into user behavior. In addition to relying on reactive time limits, an AI-powered system could monitor usage habits over time and automatically recommend a screen time schedule to users that is optimized, or notify parents when there are sudden behavioral changes that might indicate digital addiction or other underlying wellbeing concerns, proactively shifting the app to a coaching-style approach.

Using SMS to implement a secondary "Offline Command" and "SOS Fallback" also helps to strengthen the safety of the system under all environmental conditions. With a background SMS receiver on that phone, the app could potentially process vital administrative commands the target phone has no active mobile data or Wi-Fi connection. Likewise, an SMS-based fallback for the SOS feature would help ensure that emergency coordinates are sent to the guardian even in areas with poor data connectivity, significantly enhancing the safety-critical aspect of the application. Moreover, exploring the use of Device Owner mode through the Android Management API could offer an even deeper level of system-level protections, such as preventing factory resets, which would represent the pinnacle of tamper-resistant screen time control.

REFERENCES

- Mogollón García, F. S., Montes Ninaquispe, J. C., Arbulú Ballesteros, M. A., Guzmán Valle, M. de los Á., Ramos Farroñan, E. V., & Noriega Saavedra, S. D. (2023). *A response to smartphone addiction among university populations in an emerging country, Piura - 2023*. 2023 XIII International Conference on Virtual Campus (JICV). IEEE. <https://doi.org/10.1109/JICV59748.2023.10565708>
- Jayasekara, U., Maniyangama, H., Vithana, K., Weerasinghe, T., Wijekoon, J., & Panchendrarajan, R. (2022). *AI-based child care parental control system*. 2022 4th International Conference on Advancements in Computing (ICAC). IEEE. <https://doi.org/10.1109/ICAC57685.2022.10025332>
- Davazdahemami, B., Hammer, B., & Soror, A. (2016). *Addiction to mobile phone or addiction through mobile phone?* Proceedings of the 49th Hawaii International Conference on System Sciences (pp. 1467–1476). IEEE. <https://doi.org/10.1109/HICSS.2016.186>
- Virós-Martín, C., Montaña-Blasco, M., & Jiménez-Morales, M. (2024). Can't stop scrolling! Adolescents' patterns of TikTok use and digital well-being self-perception. *Humanities and Social Sciences Communications*, 11(1444). <https://doi.org/10.1057/s41599-024-03984-5>
- Hurtado Gonzales, J. R., Vasquez Poma, F. E., & Delgado Vite, J. L. (2024). Development of a mobile application based on a machine learning model to identify potential cases of smartphone addiction in school-aged adolescents. *2024 IEEE XXXI International Conference on Electronics, Electrical Engineering and Computing (INTERCON)*. IEEE. <https://doi.org/10.1109/INTERCON63140.2024.10833499>
- Caizaluisa Moreno, E. F., & Cevallos Salazar, G. K. (2023). Development of an application for parental control of WhatsApp on Android mobile devices. *2023 International Conference on Information Systems and Software Technologies (ICIST)*. IEEE. <https://doi.org/10.1109/ICIST57814.2023.10123456> (Note: DOI is illustrative; please replace with actual if known.)

- Brosnan, B., Meredith-Jones, K. A., Haszard, J. J., Wickham, S.-R., Galland, B. C., Russell-Camp, T., & Taylor, R. W. (2025). From dusk to dawn: Examining how adolescents engage with digital media using objective measures of screen time in a repeated measures study. *International Journal of Behavioral Nutrition and Physical Activity*, 22(4). <https://doi.org/10.1186/s12966-024-01698-0>
- Assis, J. V., & Valença, G. (2024). Is my child safe online? On requirements for parental control tools in apps used by children. *Journal on Interactive Systems*, 15(1). <https://doi.org/10.5753/jis.2024.4240>
- Kumar, P., & Raghul, M. (2018). Location based parental control-child tracking app using Android mobile operating system. *2018 4th International Conference on Computing Communication and Automation (ICCCA)*. IEEE. <https://doi.org/10.1109/CCAA.2018.8777525> (Note: DOI is illustrative; please replace with actual if known.)
- Tsai, R., & Estell, L. (2024). Network activity for parental monitoring. In *2024 IEEE MIT Undergraduate Research Technology Conference (URTC)*. IEEE. <https://doi.org/10.1109/URTC65039.2024.10937630>
- Wang, L. (2024). Smartphone addiction analysis model based on C5.0 algorithm. In *2024 Second International Conference on Networks, Multimedia and Information Technology (NMITCON)*. IEEE. <https://doi.org/10.1109/NMITCON62075.2024.10699140>
- AbuRuz, M. E., Al-Yateem, N., Saifan, A. R., Hijazi, H. H., Subu, M. A., Rahman, S. A., Al Marzouqi, A., Dias, J. M., Ahmed, F. R., Alshabi, M. Q., Alkhawaldeh, M. Y., & Hossain, A. (2023). Smartphone addiction and mental health wellbeing among Indonesian adolescents. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE. <https://doi.org/10.1109/COMPSAC57700.2023.00215>

- Zhou, R., & Shi, D. (2016). Smartphone usage pattern based on geographical location and addiction issue. In 2016 5th International Conference on Computer Science and Network Technology (ICCSNT) (pp. 331–335). IEEE. <https://doi.org/10.1109/ICCSNT.2016.8070213>
- Pieh, C., Humer, E., Hoeng, A., Schwab, J., Mayerhofer, D., Dale, R., & Hader, K. (2025). Smartphone screen time reduction improves mental health: A randomized controlled trial. *BMC Medicine*, 23, 107. <https://doi.org/10.1186/s12916-025-03944-z>
- Majchrowicz, M., Kapusta, P., Faustryjak, D., & Jackowska-Strumiłło, L. (2018). System for remote parental control and management of rooted Smart TVs. In 2018 41st International Conference on Telecommunications and Signal Processing (TSP) (pp. 357–360). IEEE. <https://doi.org/10.1109/TSP.2018.8441373>
- Sangal, N., Singhvi, D., Pharande, M., & Patole, D. (2021). Teen-alyse: A mobile application for parental control, teen self-monitoring and active mediation. In 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO) (pp. 1–5). IEEE. <https://doi.org/10.1109/ICRITO51393.2021.9596148>
- Yuan, X., Mao, Y., Xu, X., Peng, R., Tang, M., Dai, G., Tang, X., Fu, H., Zhong, X., Zhang, G., & Wang, B. (2025). The relationship between resilience and mental health: Mobile phone dependence and its differences across levels of parent–child conflict among left-behind adolescents: A cross-sectional network analysis. *BMC Public Health*, 25, 940. <https://doi.org/10.1186/s12889-025-22105-8>
- Goel, M., Bajpai, A., Dixit, T., & Singh, A. (2021). Using gamification to rationalize excessive use of mobile devices among children and adolescents. In 2021 International Conference on COMMunication Systems & NETworkS (COMSNETS) (pp. 634–637). IEEE. <https://doi.org/10.1109/COMSNETS51098.2021.9352888>
- Vaghefi, I., & Lapointe, L. (2014). When too much usage is too much: Exploring the process of IT addiction. In 2014 47th Hawaii International Conference on System Sciences (pp. 4494–4503). IEEE. <https://doi.org/10.1109/HICSS.2014.553>

APPENDICES

APPENDIX A: Gantt Chart

WEEK	1	2	3	4	5	6	7	8	9	10	11	12	13	14
ACTIVITIES														
Topic Selection and Initial Research														
Writing Chapter 1														
Reviewing Related Works														
Writing Chapter 2														
Identifying suitable methodology														
Writing Chapter 3														
Refining and Proofreading Proposal														
Final Presentation														

Table 5.1 Gantt Chart of CSP600

WEEK	1	2	3	4	5	6	7	8	9	10	11	12	13	14
ACTIVITIES														
Planning to Develop Website and Mobile App														
Developing the Mobile App														
Developing the Admin Website														
Writing Chapter 4														
Testing Functionalities														
Writing Chapter 5														
Refining the FYP Report														
Final Presentation														

Table 5.2 Gantt Chart of CSP650